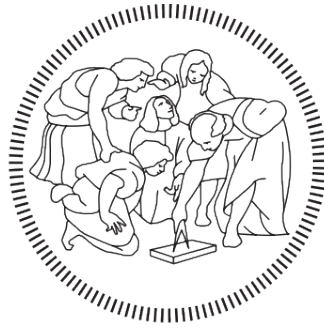




POLITECNICO

MILANO 1863

054323 – Software Engineering 2 2025
RASD

Requirement Analysis and Specification
Document

Best Bike Paths

Juan Sebastian Sarmiento Pulido- 11153648
Pedro Augusto Villarinho dos Santos - 11108811
Esteban Lopez Barreto - 11154567

Tuesday 23rd December, 2025

Contents

1	INTRODUCTION	3
1.1	Purpose	3
1.1.1	Goals	3
1.2	Scope	3
1.2.1	World Phenomena	3
1.2.2	Shared Phenomena	4
1.3	Definitions, Acronyms, Abbreviations	4
1.3.1	Definitions	4
1.3.2	Acronyms	5
1.3.3	Abbreviations	5
1.4	Revision history	5
1.5	Reference Documents	6
1.6	Document Structure	6
2	OVERALL DESCRIPTION	6
2.1	Product perspective	6
2.1.1	Scenarios	6
2.1.2	Domain class diagram	8
2.1.3	State Diagrams	9
2.2	Product functions	10
2.3	User characteristics	11
2.4	Assumptions, dependencies and constraints	12
2.4.1	Regulatory policies	12
2.4.2	Domain Assumptions	12
3	SPECIFIC REQUIREMENTS	12
3.1	External Interface Requirements	12
3.1.1	User Interfaces	12
3.1.2	Hardware Interfaces	21
3.1.3	Software Interfaces	21
3.1.4	Communication Interfaces	22
3.2	Functional Requirements	22
3.2.1	Use Case Diagram	25
3.2.2	Use Cases	27
3.2.3	Sequence Diagrams	38
3.2.4	Requirement Mapping	49
3.3	Non-functional requirements	59
3.3.1	Security	59
3.3.2	Compatibility	59
3.3.3	Scalability	59
3.3.4	Performance	59
3.3.5	Availability	59

3.3.6	Interoperability	60
3.4	Design Constraints	60
3.4.1	Standards compliance	60
3.4.2	Hardware limitations	60
3.4.3	Any other constraint	60
3.5	Software System Attributes	61
3.5.1	Reliability	61
3.5.2	Availability	61
3.5.3	Security	61
3.5.4	Maintainability	61
3.5.5	Portability	62
4	FORMAL ANALYSIS USING ALLOY	62
4.1	Alloy Code	63
4.2	Scenarios	71
5	EFFORT SPENT	77
6	REFERENCES	77

1 INTRODUCTION

1.1 Purpose

Cyclists often face the challenge of finding a safe and well-maintained route for their journeys. Information about bike paths is often incomplete and scattered through multiple sources such as online forums and even word-of-mouth, making it difficult for riders to plan their trips and avoid obstacles such as potholes or poorly maintained roads. In addition to that, cyclists also lack accessible tools to track their own performance while doing these trips. The purpose of BBP is to address all these issues by providing an integrated platform where users can record, analyze, and share their cycling experience. Through user driven data collection, BBP should be able to collect detailed information on roads, road conditions, and performance metrics, ensuring a community-driven system that promotes safety, collaboration, and smarter urban cycling.

1.1.1 Goals

G1: Cyclists would be able to record the bike routes they use, along with any obstacles they encounter.

G2: Cyclists would be able to keep track of the bike paths they take.

G3: Cyclists would be able to view detailed information and statistics for the bike routes they have taken.

G4: Cyclists would be able to publish their bike paths and their reviews of other bike paths so that other cyclists can see and use them.

G5: Cyclists would like to be able to search for optimal and well-maintained bike paths by specifying an origin and a destination.

G6: Cyclists would like the most up to date information regarding a bike path according to other cyclists' experiences.

1.2 Scope

The aim of this project is to support cyclists in discovering, recording, and sharing bike paths, giving them tools for the creation of highly detailed cycling routes as well as a platform where user's submissions help keep an updated inventory of them. Users must provide information about their rides either manually or automatically, in the latter case, using the automatic path recognition mechanisms offered by the platform, which identify all kinds of information, which must then be confirmed by the user .

The platform will facilitate the search, visualization and dissemination of routes among different users, through the platform selection of the optimal routes, based on the published routes metrics and user confirmation and new experiences, getting a final result that will adapt to the changes in the environment noticed by the users.

1.2.1 World Phenomena

- WP1: A cyclist takes a bicycle trip.

- WP2: The condition of a path changes over time, for example, as new irregularities appear.
- WP3: Weather conditions differ from trip to trip and change over time.

1.2.2 Shared Phenomena

Environment controlled

- SP-EC1: The user manually inserts a bicycle trip in the BBP platform.
- SP-EC2: The user makes public the information about a bike path.
- SP-EC3: Information about a path is collected from the user's mobile device GPS, accelerometer, and gyroscope
- SP-EC4: The user revises the information collected (Automatically) by the BBP platform about a bike route taken by the user
- SP-EC5: The user visualizes the bike paths between an origin and a destination point.

Machine controlled

- SP-MC1: Detailed information and statistics of a bicycle trip taken are available to the user through the BBP platform.
- SP-MC2: BBP platform shows the reconstructed path taken to the user
- SP-MC3: BBP platform shows the user the obstacles found in the path taken
- SP-MC4: BBP platform exposes the optimal and well-maintained paths for an origin and a destination point to the user.

1.3 Definitions, Acronyms, Abbreviations

1.3.1 Definitions

- Bike Paths: A route recorded and defined by a user while cycling, intended to follow streets or trails where a proper bike track exists, or where motor traffic is minimal and speed limits are compatible with typical cycling speeds.
- Gyroscope: A sensor that measures the orientation and angular velocity of the phone.
- GPS: A satellite-based navigation service that provides accurate information about the geographic location, velocity, and time of a device anywhere on Earth.
- Accelerometer: A sensor that measures the acceleration of the phone.
- Public Path: A Bike Path recorded and submitted to the platform that is accessible to all users.

- Status: A categorical label describing the overall condition of a Bike Path (Optimal, Medium, Sufficient, Requires maintenance).
- Obstacles: Physical elements along a bike path that may affect cycling, such as potholes, bumps, debris, construction sites or others.
- Statistics: Metrics associated with a recorded Bike Path and shown in the user's records; at minimum these include total distance and duration, and the system may include additional computed metrics derived from the recorded Bike Path when available such as average speed, maximum speed, elevation gain and weather conditions.
- Path Score: A numerical value used to rank Bike Paths for search results, reflecting route condition and efficiency based on available updates.
- Segment: A contiguous portion of a Bike Path associated with a specific street (or street name).
- Weather Conditions: Environmental data attached to an automatically recorded Bike Path when a Weather Service is available, including sunny, cloudy, rainy, windy, stormy and snowy.

1.3.2 Acronyms

- BBP: Best Bike Path
- GPS: Global Positioning System

1.3.3 Abbreviations

- G: Goal
- WP: World phenomena
- SP-EC: Environment controlled shared phenomena
- SP-MC: Machine controlled shared phenomena
- UC: Use Case

1.4 Revision history

- Version 0.1: Setup
 - Created first layout.
- Version 1.0: First release
 - Section 1 added.
 - Section 2 added.
 - Section 3 added.
 - Section 4 added.

1.5 Reference Documents

The assignment for this document and all the information included refer to the following documentation:

- The specification for the 2025/26 Requirement Engineering and Design Project for the Software Engineering II course.
- The slides on the webeep page of the Software Engineering II course.
- Alloy documentation: <https://alloytools.org/documentation.html>

1.6 Document Structure

1. **Introduction:** A short overview of the project derived from the given specification, outlining the product's main objectives and overall scope.
2. **Overall Description:** A more comprehensive product description that incorporates real-world situations and evaluates the system from a technical standpoint.
3. **Specific Requirements:** A complete set of the requirements required to achieve the goals, together with their connections to the underlying domain assumptions.
4. **Formal Analysis Using Alloy:** A formal specification of selected real-world phenomena modeled in Alloy to reveal possible inconsistencies between the assumptions and the previously defined phenomena.
5. **Effort Spent:** The time each member of the team invested in creating this document
6. **References:** References to materials and tools employed during the development of this document.

2 OVERALL DESCRIPTION

2.1 Product perspective

2.1.1 Scenarios

- **User registers to the BBP platform**

The user intends to create an account in the BBP platform. They go to the log in page from the homepage, and then to the registration page. There, they insert the informations required to register and confirm.

- **User logs in to the BBP platform**

The already registered user wants to log in to the BBP platform. From the homepage, they go to the log in page. There, they insert their e-mail and password. If the data matches, they will be sent to the homepage, already logged in, if not, they will be asked to try again with different credentials.

- **User manually records a path**

The already registered user intends to contribute to the BBP platform with a path. They log in to BBP, and, having in mind all the specifications, of the path they choose to manually insert a path. They indicate the names of the streets that belong to this path, as well as the status of the path and finish the operation.

- **The BBP platform records a path**

The already registered user intends to contribute to the BBP platform with a path. The user brings their mobile device on a bike ride. Before they start the ride, they log in to BBP and begin their automatic recording. When the user is done, they stop recording their activity, and the path is saved.

- **User visualizes all the bike rides recorded (Keep track)**

The already registered user intends to visualize their history of bike rides. They log in to BBP and go to the records page. On this page, there are all the manual and automatic recordings of their bike rides with all their available information.

- **User visualizes detailed information (statistics)**

The already registered user intends to see their own statistics available on the BBP platform. They log in to BBP and go to the records page, where they can visualize all the bike trips. When clicking on any of them, the user is able to see more detailed information about that specific trip.

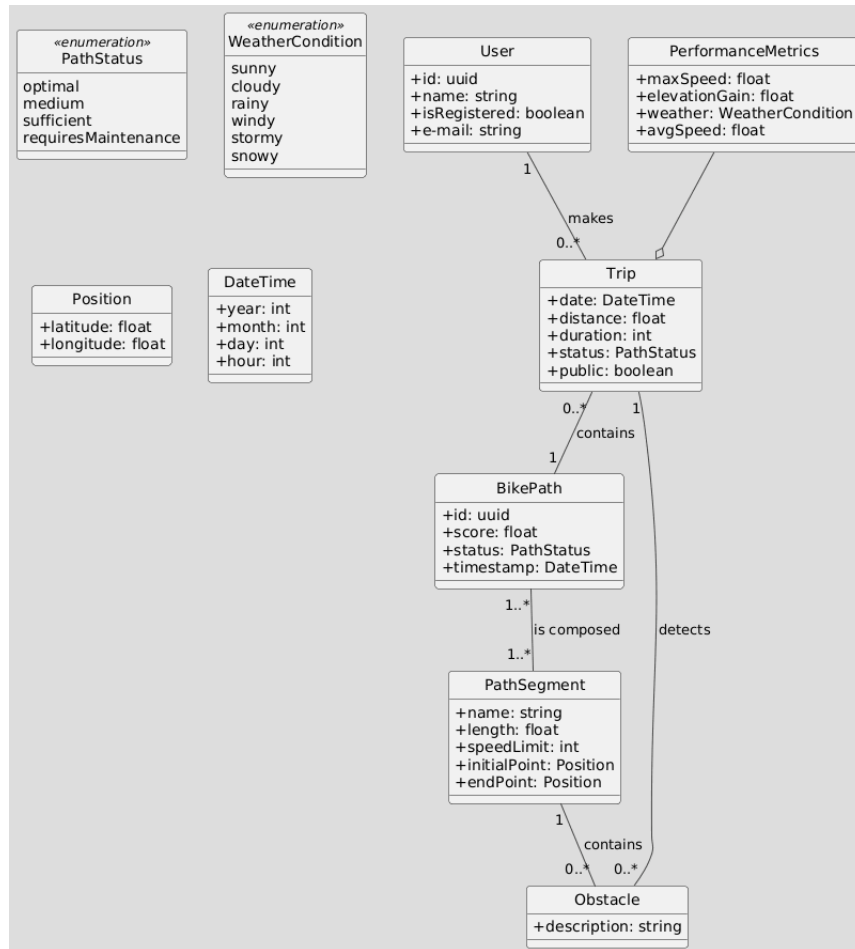
- **User makes public one of their paths to share with (Status - obstacles) (Manually and confirm for automatically recording)**

The already registered user intends to share their bike path with other users of the BBP platform. They log in to BBP, and go to the records page. Then, the user should select any path that they would want to publish. Then, upon visualizing the details of the path, they can select and confirm that they want to make it public.

- **User visualizes a path based (optimal, well-maintained) on a search (with origin and destination) (Get by feedback of recent users)**

The already registered user wants to search for a bike path. They log in to BBP and insert the origin and destination names of the path they are searching for. If there is a recorded path with these points in the BBP platform, the user is lead to the map service, where they can see the most optimal and well-maintained trajectory that leads from one point to the other, specifying also the status and obstacles that the path may have.

2.1.2 Domain class diagram



This Domain Class Diagram represents the main entities of BBP and how route information is structured and reused across users. The model is centered on **BikePath** as the primary “route” artifact, while **Trip** mainly captures a single user’s recorded ride.

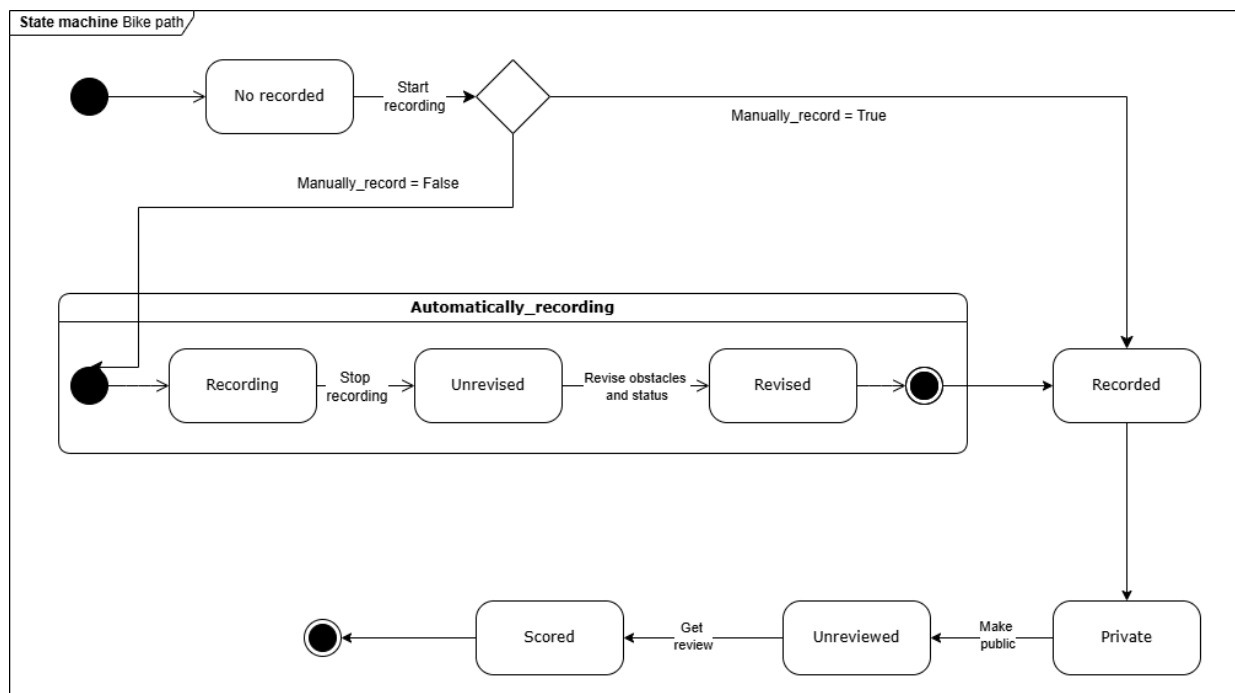
- **BikePath** is the core domain object used to represent a route in the platform: it has an id, a score, a status (**PathStatus**), and a timestamp, so it can be ranked and presented consistently in searches and views.
- A **BikePath** is composed of **PathSegments**, where each **PathSegment** stores road-level properties (name, length, speedLimit, and **PathType**), making the route a structured sequence rather than just raw GPS points.
- Obstacles are associated with **PathSegments** (not directly with **Trips**): each **Obstacle** has a type, a geographic **Position** (latitude/longitude), and a **DateTime** timestamp, so the same obstacle knowledge can be reused by any **BikePath** that includes that segment.

- A Trip represents one ride performed by a User and stores the “ride outcome” fields (date, distance, avgSpeed, duration, status, and whether it is public), plus optional PerformanceMetrics (calories, maxSpeed, elevationGain).
- The key distinction is that Trips are individual recordings, while BikePaths are the reusable, shareable route entities built from segments and obstacles; in other words, Trips provide data, but BikePaths are what the system should primarily “talk about” when searching, scoring, and showing routes.
- Weather is modeled as a categorical enumeration (WeatherCondition: sunny, cloudy, rainy, windy, stormy, snowy), which can be attached to recorded information to contextualize a Trip and, indirectly, support interpretation of a BikePath’s condition.

Note: From this point onward, although the Trip entity is still present in the model (to represent a single recorded ride and its related performance data), the document will mainly use the Bike Path concept as Bike Path is the primary domain object for route sharing, searching, scoring, and obstacle/status aggregation, so the following requirements and descriptions will be expressed mostly in BikePath terms.

2.1.3 State Diagrams

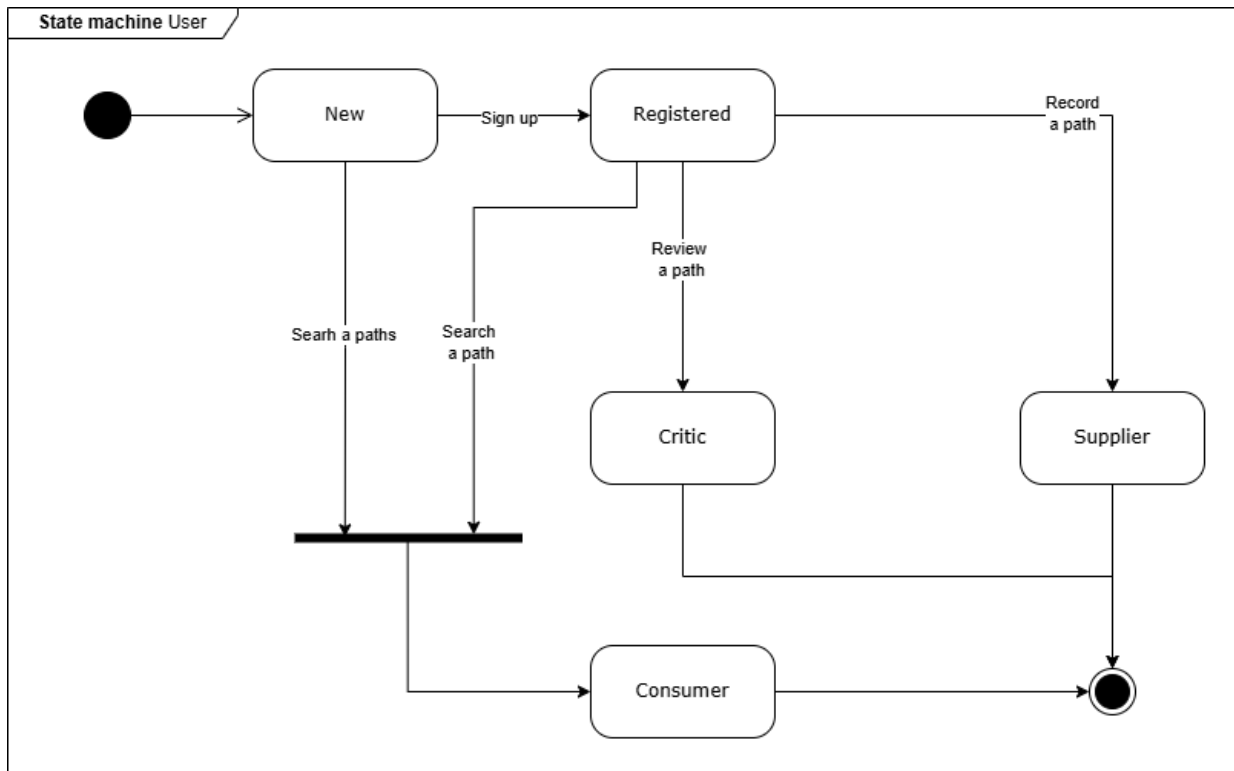
Bike Path



The state machine diagram above describes the complete lifecycle of a bike path within the BBP platform, from its creation to its review by other users. Initially, a bike path is not recorded; once the recording process starts, the system distinguishes between manual

and automatic recording. In the manual case, the bike path is directly stored, reaching the "Recorded" node. In the automatic case, the system enters the Automatically_recording phase in which the user must review and confirm detected obstacles and path status to ensure data reliability, after they stop recording. Then, the bike path is marked as recorded and remains private by default. The user may choose to make it public, at which point the path becomes visible but initially unreviewed. As other users provide feedback, the bike path is evaluated and assigned a score, reaching its final state.

User



The User state machine diagram describes the progress of a user's role within the BBP platform. Initially, a user enters the system in the "New" state and may search for bike paths without being registered. By completing the sign-up process, the user transitions to the "Registered" state, which enables additional functionalities. A registered user can record a bike path, becoming also a "Supplier", or review existing paths, becoming a "Critic". In the end, the user will always act as a "Consumer" of the available bike path information and the BBP platform overall. This model highlights that while all users can search and consume path data, only registered users can actively contribute to the recording and reviewing functionalities.

2.2 Product functions

- Log in and register

To register in the BBP system, it is required the user's name, e-mail, and a password they would like to use. The login details will be correctly encrypted and safely maintained in the system's database once the user completes their registration. To log in, the user should press the log in button on the home page, insert their e-mail and password, and confirm. Once their account login info is authenticated, the user is sent to the home page, already logged in.

- Managing of paths

It is possible for a logged in user to make a path public for other users to be able to see and review it. To do this they should go to the "My records" page, select the path they desire and publish it. Still in the same page, they can see the list of their recorded paths and visualize their respective information. The user can also review other user's paths once they search them, by going to the review page, filling in with the information required and posting it.

- Recording of paths

To record a path, the logged in user should click on the record button when their are in the homepage. After that they can choose between a manual or an automatic insertion. If the user chooses manual, they need to fill in with the information about the path and confirm. For the automatic option, the BBP will take information from the GPS Service, as well as the Sensor Service and Weather Service of the user's device, inserting all the information in the path. On both options, the system will finally insert the confirmed path into the user's "My Records" page. Also on the automatic recording page, the user is able to revise obstacles on a certain segment, editing out possible insertion mistakes of obstacles into the system.

- Searching of paths

To search a path, the logged in user should fill in the origin and destination textboxes with the desired locations, and the BBP system will try to match them with the database. If there is matching data, the system will provide a path that links the two points, giving details about the segments' obstacles and the path's status overall, as well as showing the map of the searched path. If the user is not satisfied with the given path, they can view alternative routes in the same page by pressing the "View more routes" button.

2.3 User characteristics

The application is designed for any person who wants to keep track of the bike routes they take by recording them anywhere and viewing them whenever they wish. It also supports anyone who wants to search for optimal and well maintained routes from one point to another, and those who want to collaborate with the community by discovering, rating, and ranking different bike paths to improve the experience for others.

For these reasons, the potential user base includes any cyclist, meaning anyone who uses a bicycle, whether for everyday travel or simply for fun. Users should be able to interact

with the application and, ideally, have a smartphone with multiple sensors that can collect movement data while they ride.

2.4 Assumptions, dependencies and constraints

2.4.1 Regulatory policies

The BBP platform collects and processes specific categories of user information necessary for the correct operation of its services. The data required from users includes e-mail address, name, password, live location data, live performance metrics, and live weather conditions.

All personal data collected by BBP is processed exclusively for the purposes described above and in full compliance with the General Data Protection Regulation (GDPR) and applicable privacy legislation.

2.4.2 Domain Assumptions

- D1: Users report the bike path status truthfully.
- D2: Users report obstacles observed on the bike path truthfully.
- D3: The User's mobile device has a fully operative GPS.
- D4: The User's mobile device has a fully operative accelerometer and gyroscope sensors.
- D5: The User does not use a "Fake GPS" while recording a path.
- D6: Path's origin and destination have traceable addresses.
- D7: Each segment of the bike path taken by the user have traceable address.
- D8: Bike paths are where a proper bike path exists.
- D9: Bike paths are where cars are rare and speed limits are compatible with the average speed of a bike.
- D10: All users have a reliable internet connection.

3 SPECIFIC REQUIREMENTS

3.1 External Interface Requirements

3.1.1 User Interfaces

Here is shown a mockup of the web application UI. The images depict the main screens of the system, which include a login and registration interface, a home page where users can search bike paths by origin and destination even without an account, once logged it

has dedicated sections for recording trips, either manually or automatically, with options available to edit both manual and automatic records like adding streets and/or obstacles. The application also provides a page for visualizing all recorded paths and another for reviewing individual trips shared by other people.

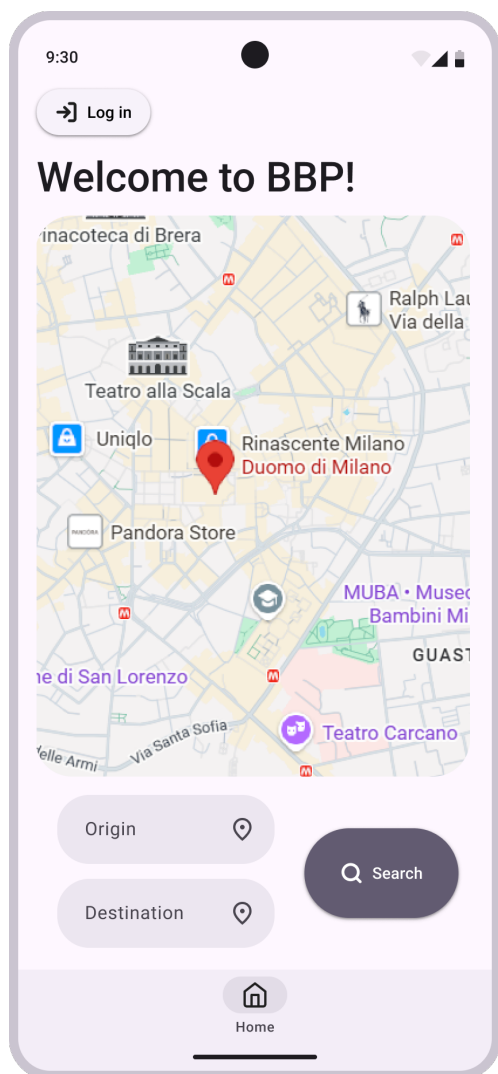


Figure 1: HomePage Not LoggedIn.

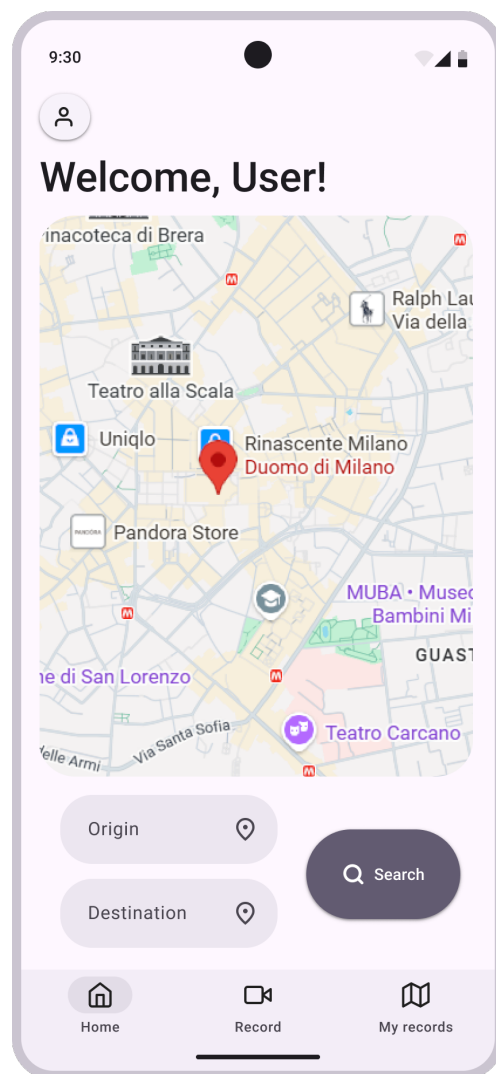
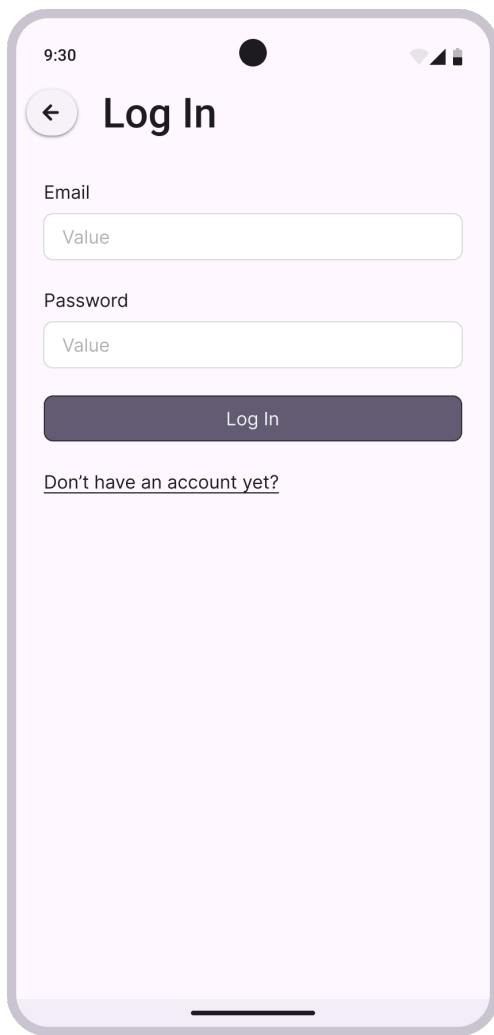


Figure 2: HomePage LoggedIn.



A mobile app login screen with a light purple background. At the top, the status bar shows 9:30, a black circle, and signal icons. Below the status bar is a header with a back arrow in a circle and the text "Log In". The main content area contains two input fields: "Email" and "Password", each with a light green placeholder text "Value". Below these fields is a dark purple button with the text "Log In". At the bottom, there is a link that says "Don't have an account yet?". The bottom of the screen features a home indicator bar.

9:30

← Log In

Email

Value

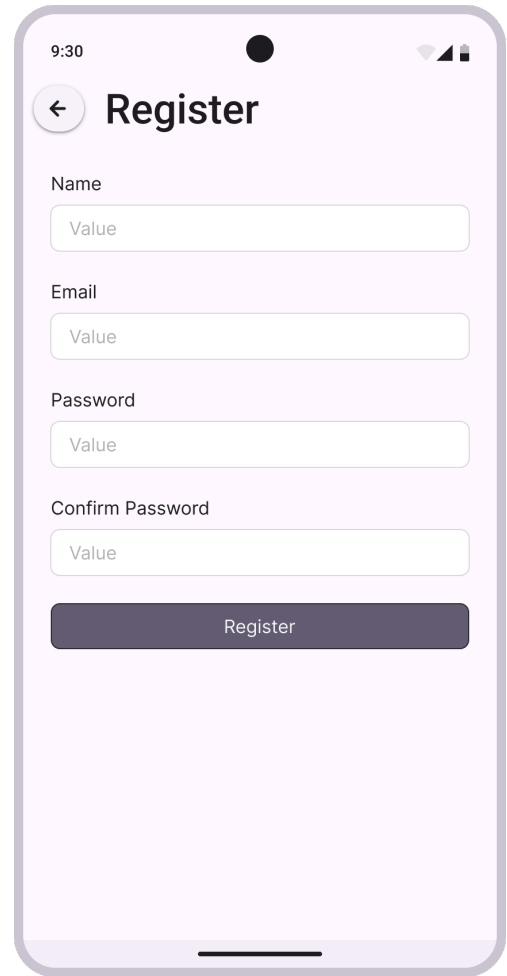
Password

Value

Log In

[Don't have an account yet?](#)

Figure 3: Login Page.



A mobile app register screen with a light purple background. At the top, the status bar shows 9:30, a black circle, and signal icons. Below the status bar is a header with a back arrow in a circle and the text "Register". The main content area contains four input fields: "Name", "Email", "Password", and "Confirm Password", each with a light green placeholder text "Value". Below these fields is a dark purple button with the text "Register". The bottom of the screen features a home indicator bar.

9:30

← Register

Name

Value

Email

Value

Password

Value

Confirm Password

Value

Register

Figure 4: Register Page.

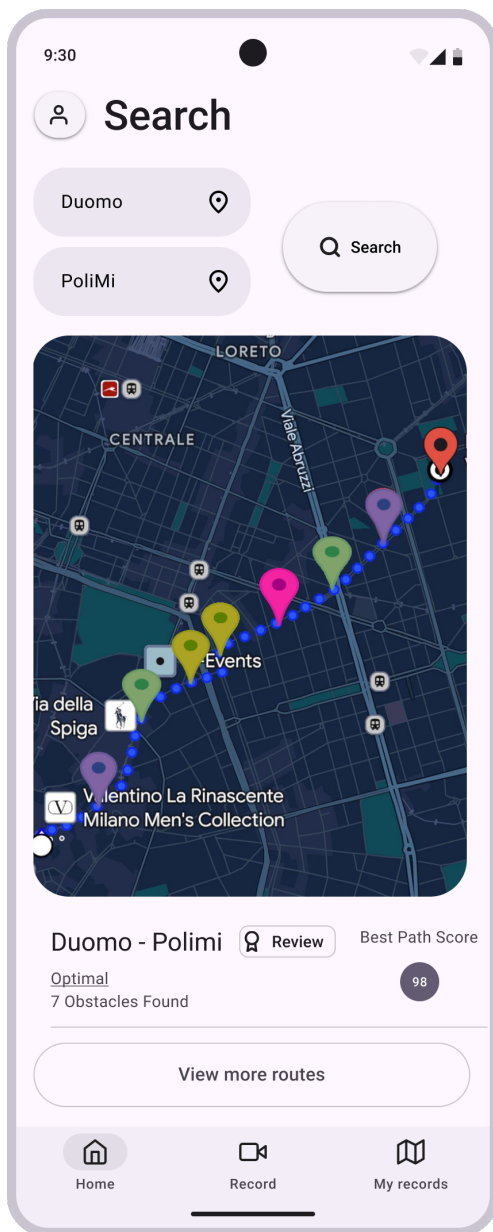


Figure 5: Search page.

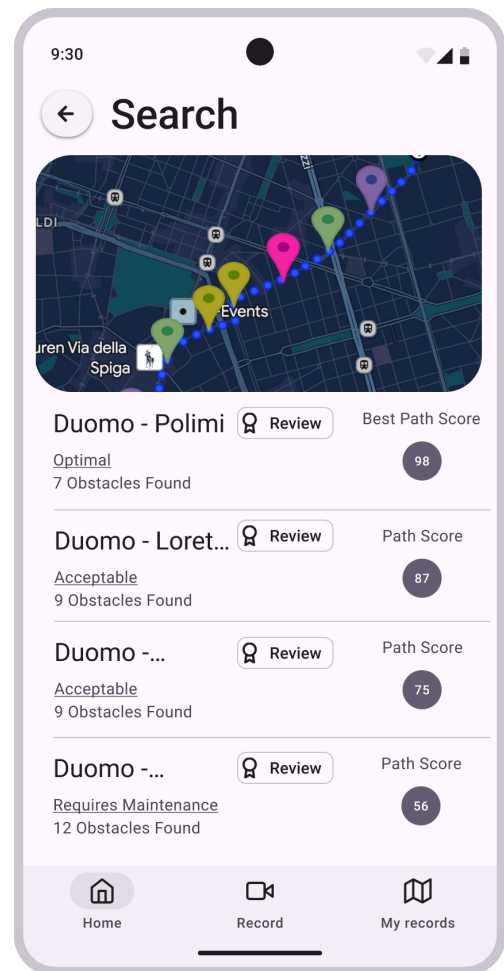


Figure 6: More routes page.

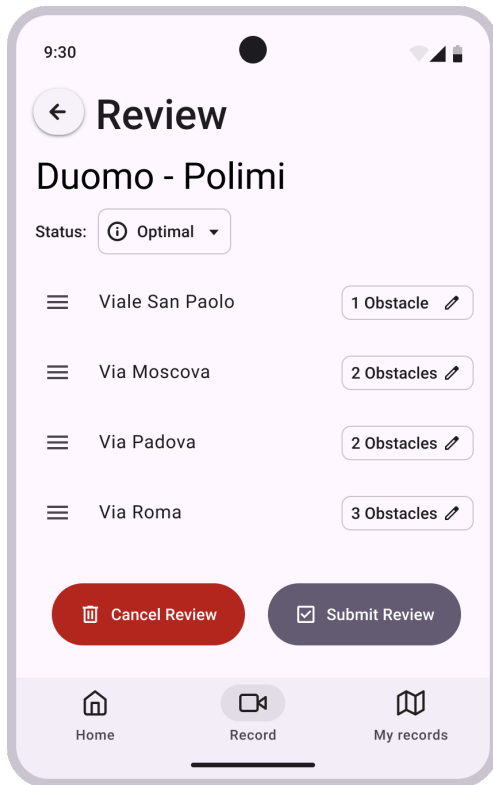


Figure 7: Review page.

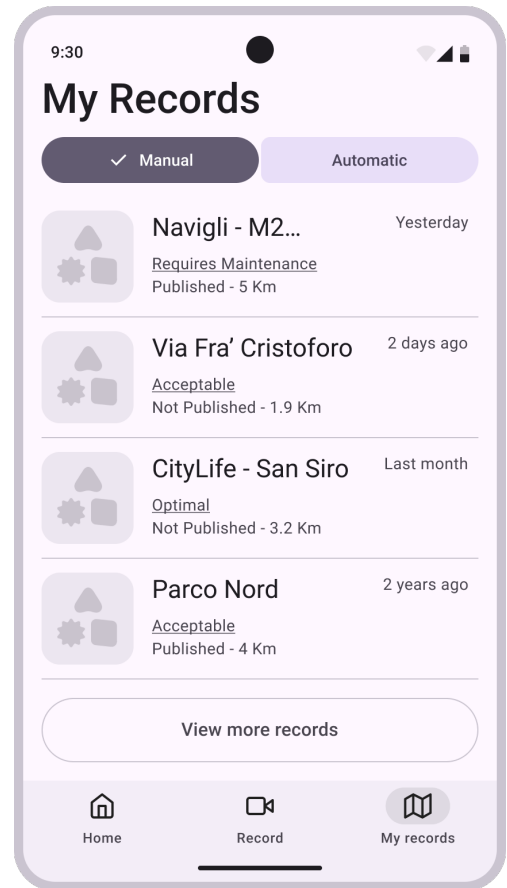


Figure 8: My Manual Records page.

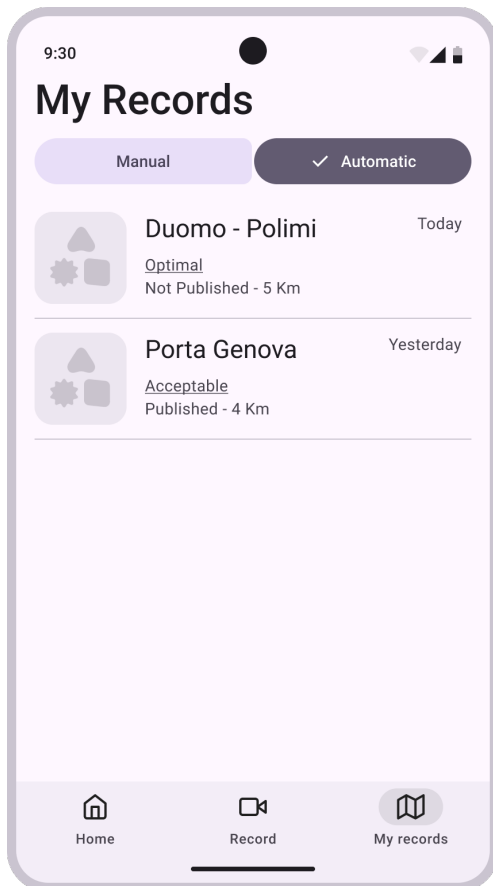


Figure 9: My Automatic Records page.

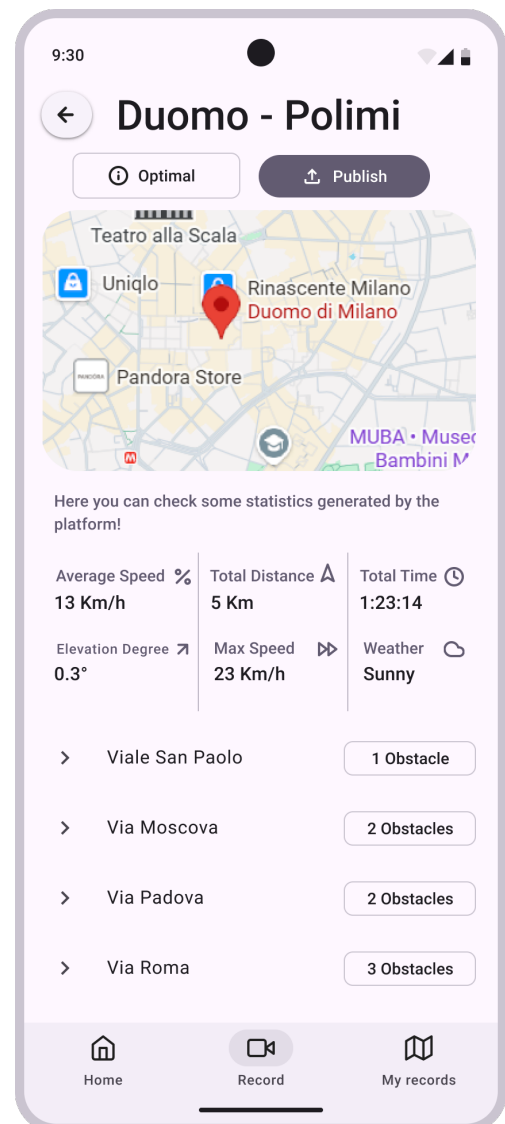


Figure 10: My record page.

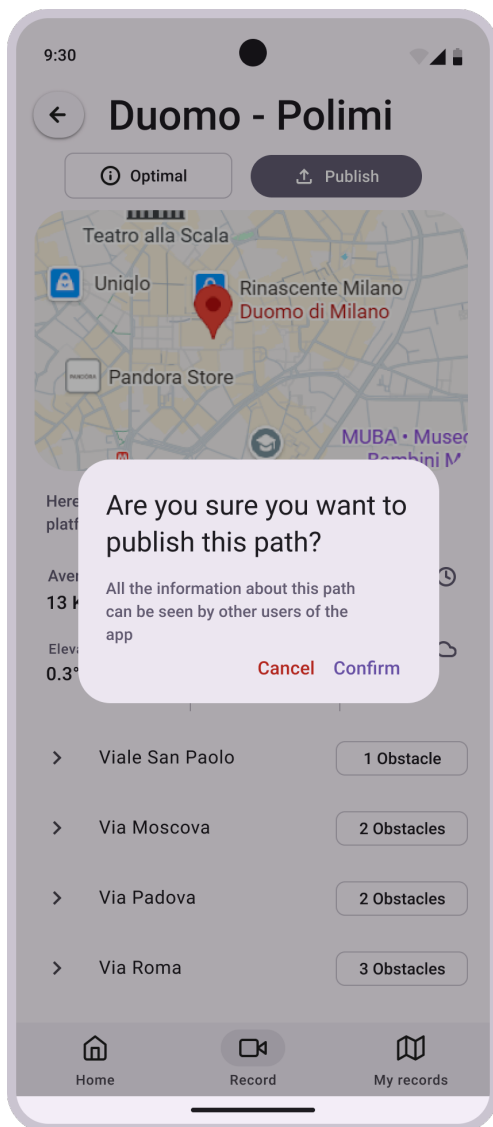


Figure 11: Publish window.

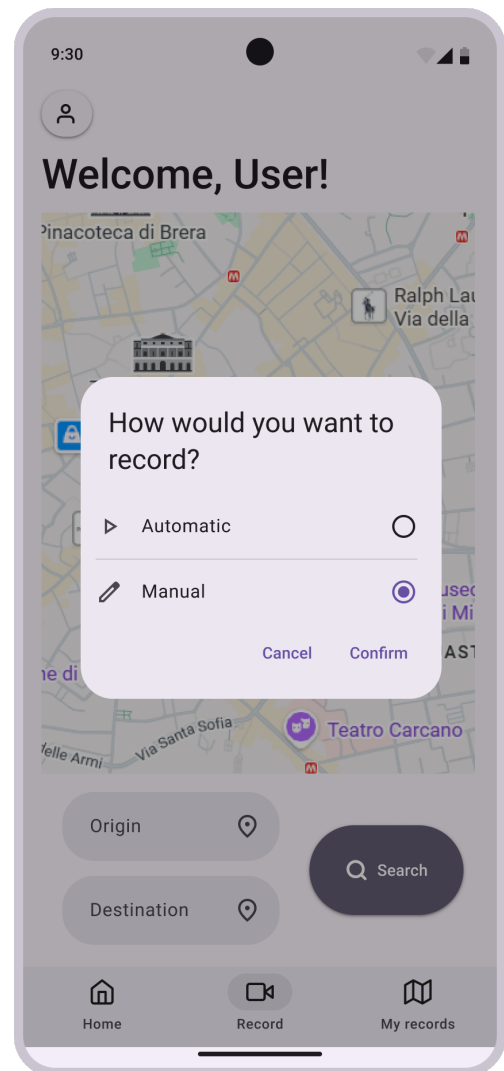


Figure 12: Record window.

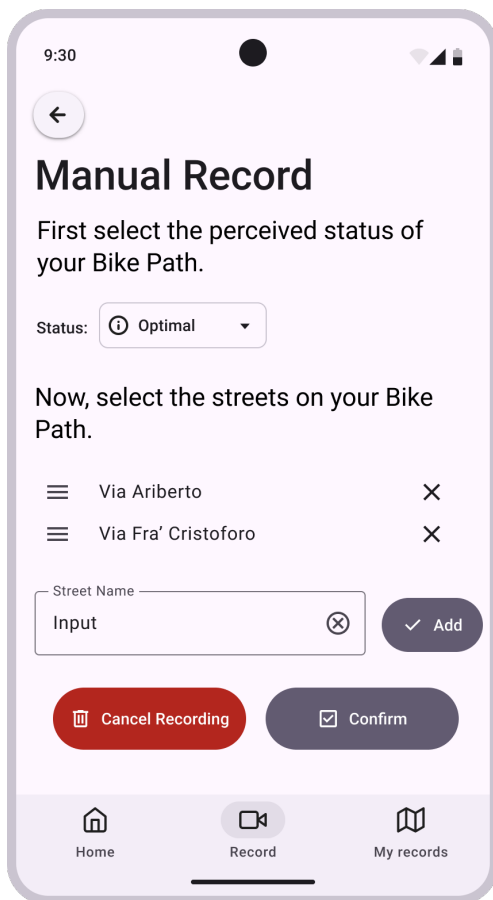


Figure 13: Manual record page.

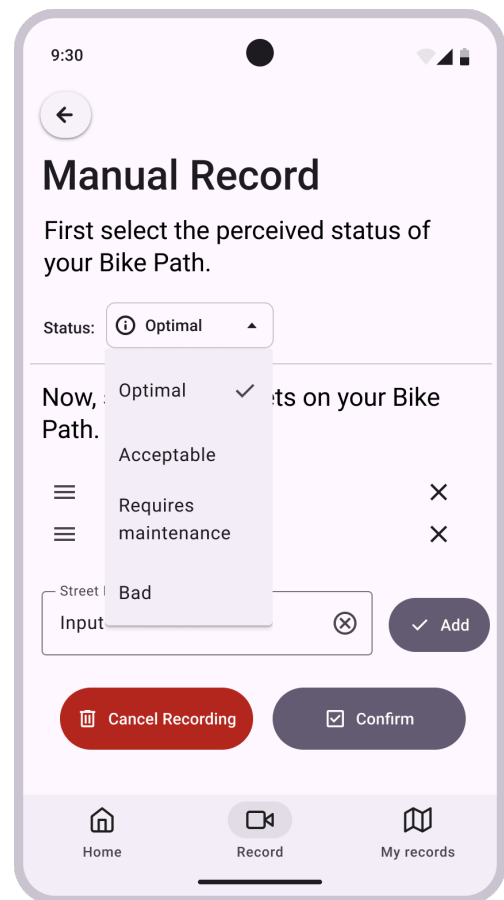


Figure 14: Status dropdown menu.

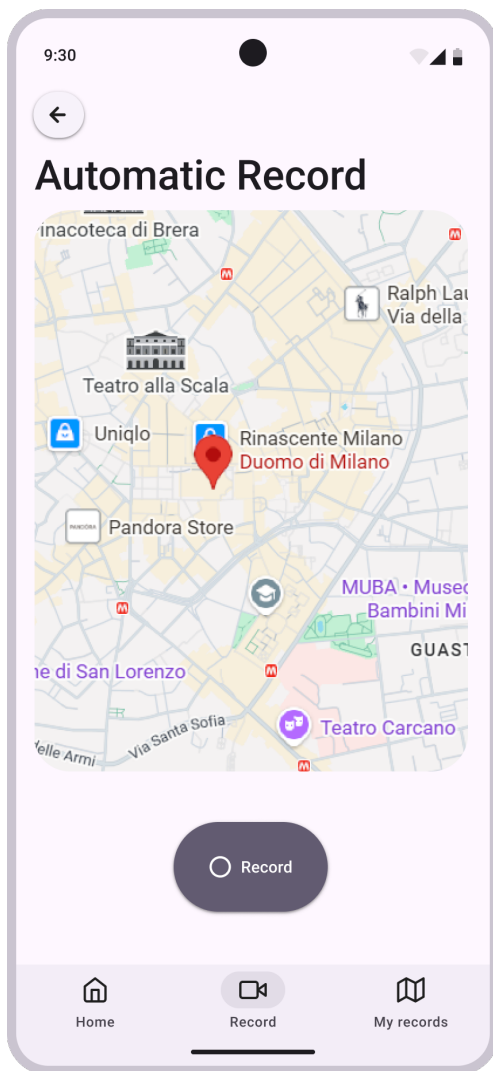


Figure 15: Automatic Record page.

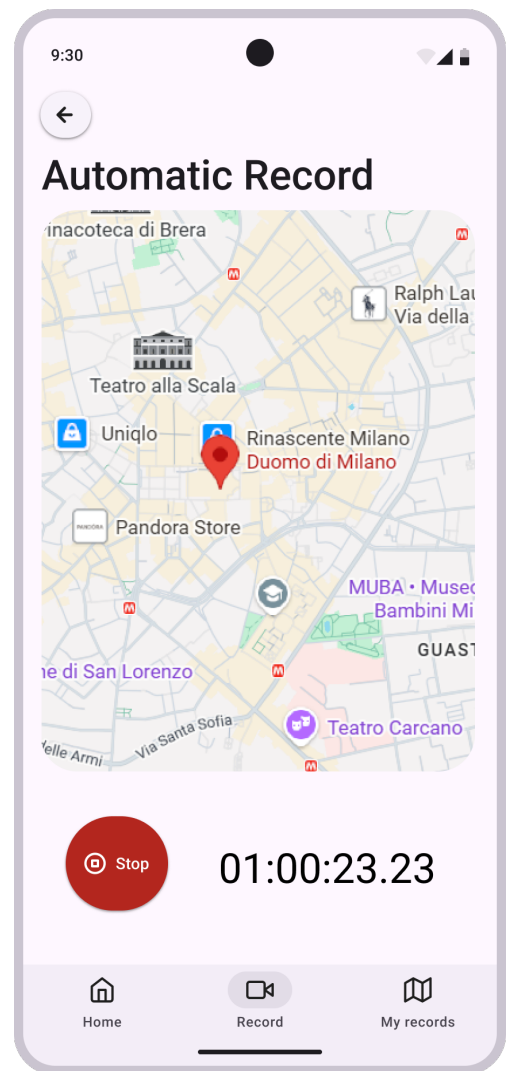


Figure 16: Recording page.

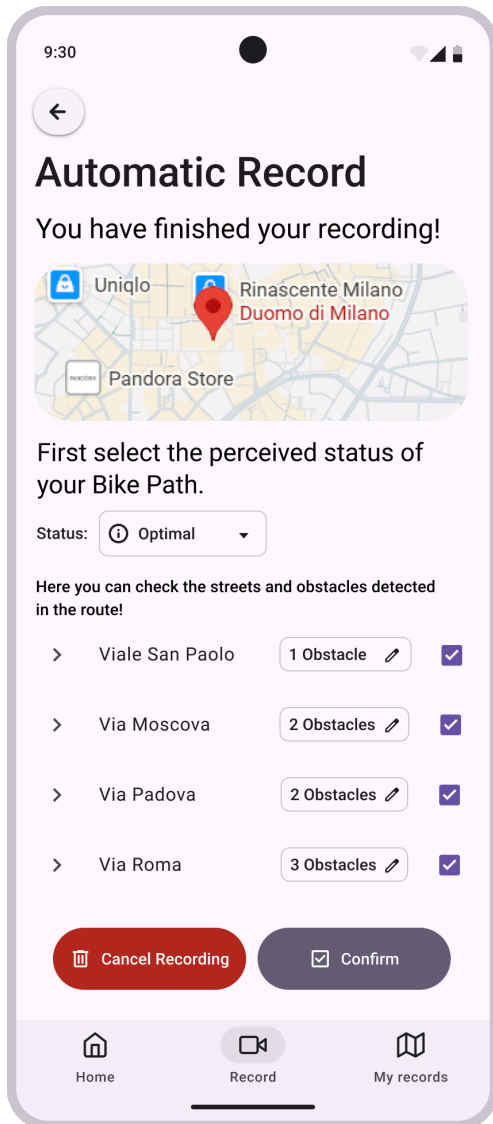


Figure 17: Recording Finish page.

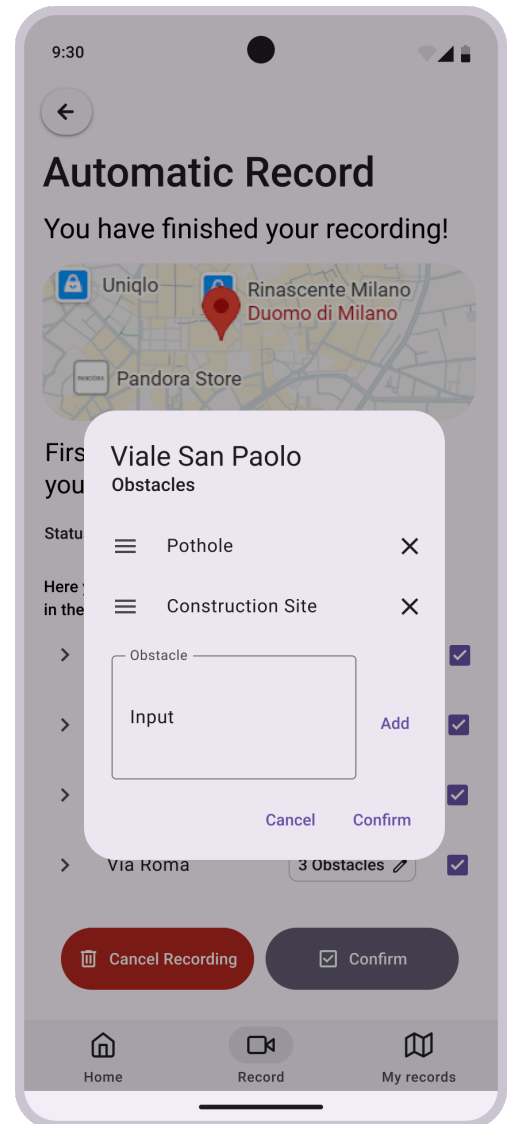


Figure 18: Obstacle Window menu.

3.1.2 Hardware Interfaces

BBP is a web-based platform designed primarily for mobile devices, as it relies on sensors such as the gyrometer, accelerometer, and other motion or location sensors typically available only on smartphones. While it can be accessed from any device with a web browser and an internet connection, full functionality is available only on mobile devices due to the need for these built-in sensors. No additional external hardware is required.

3.1.3 Software Interfaces

- **GPS Service Interface:** The system relies on an external GPS service, such as Google Maps or a similar provider, to retrieve location, routing, and path-tracking information. This interface enables automatic trip recording, origin-destination queries,

and geographic visualization within the platform.

- **Weather Service Interface:** To enrich recorded trips with environmental context, the application connects to a weather forecast service capable of supplying data such as temperature, wind speed, and weather conditions at the time of the trip. This information is integrated into the trip records to enhance statistics and analysis.

3.1.4 Communication Interfaces

Since the system is a web application, all interactions with users must occur securely over the internet. All Data transmission must be encrypted using HTTPS. User authentication is handled through the platform login and registration system. Communication with external services, such as the GPS provider (ex. Google Maps) and the weather forecast provider, is performed through authenticated HTTP requests.

3.2 Functional Requirements

Log in and register

- R1: When an unregistered user enters the BBP platform, it shall allow the user to create an account.
- R2: When a registered user enters the BBP platform, it shall allow them to log into their account.

Recording of paths

- R3: The BBP platform shall allow a logged-in user to manually record a bike path by providing the names of the streets on the bike path and their status.
- R4: When a logged-in user enters the BBP platform, it shall allow them to automatically record a bike path they are taking.
- R5: When a logged-in user automatically records a bike path, the BBP platform shall recognize obstacles on the path.
- R6: When a logged-in user with automatically recorded bike paths enters the BBP platform, it shall allow them to confirm the presence of obstacles automatically recognized in the recorded process.
- R7: When a logged-in user with automatically recorded bike paths enters the BBP platform, it shall allow them to correct automatically recognized obstacles in the path.

Managing of paths

- R8: The BBP platform shall show to a logged-in user their list of recorded paths.

- R9: The BBP platform shall provide a logged-in user the path statistics about their automatically recorded paths.
- R10: When the Weather service is available, the BBP platform shall provide weather information to a logged-in user for their automatically recorded paths.
- R11: When a logged-in user with recorded paths enters the BBP platform, it shall allow them to make their bike paths visible to other users who are searching for routes.
- R12: The BBP platform shall allow a logged-in user to insert new status and obstacles information to a public bike path.
- R13: When a logged-in user selects a previously recorded bike path, the BBP platform shall display a map view showing the selected bike path and its associated obstacles.
- R14: When a logged-in user selects a previously automatically recorded bike path, the BBP platform shall display the path statistics computed in the recording process for the selected bike path.
- R15: When a logged-in user selects a previously automatically recorded bike path that has weather information, the BBP platform shall display the stored weather information for the selected bike path.
- R16: When a logged-in user selects a previously recorded bike path, the BBP platform shall display the list of streets traversed by the selected bike path in the stored segment order.
- R17: When a logged-in user selects a previously recorded bike path, the BBP platform shall display the obstacles associated with the selected bike path showing for each obstacle its type and the street where it was detected.

Searching of paths

- R18: When a user enters the BPP platform, it shall allow the user to search bike paths considering an origin and a destination point.
- R19: When a user performs a bike-path search and the BBP platform does not have recorded bike paths that satisfy the search parameters, the BBP platform shall generate candidate bike paths by combining recorded segments (including their associated obstacles) from existing bike paths.
- R20: The BBP platform shall display bike paths options (First one on a map), corresponding to an user search (with the origin and destination point) in descending order depending on their path score.

Path information collection and merging

- R21: When the BBP platform receives new status and obstacle information from a public bike path, it shall update their path score according to all updates made to the path information, prioritizing the most up-to-date information.
- R22: When the BBP platform generates new bike paths in response to a user bike-path search, the BBP platform shall estimate and assign an overall status to each generated bike path, deriving that status from the statuses of the existing bike paths that include the segments used to form it.

3.2.1 Use Case Diagram

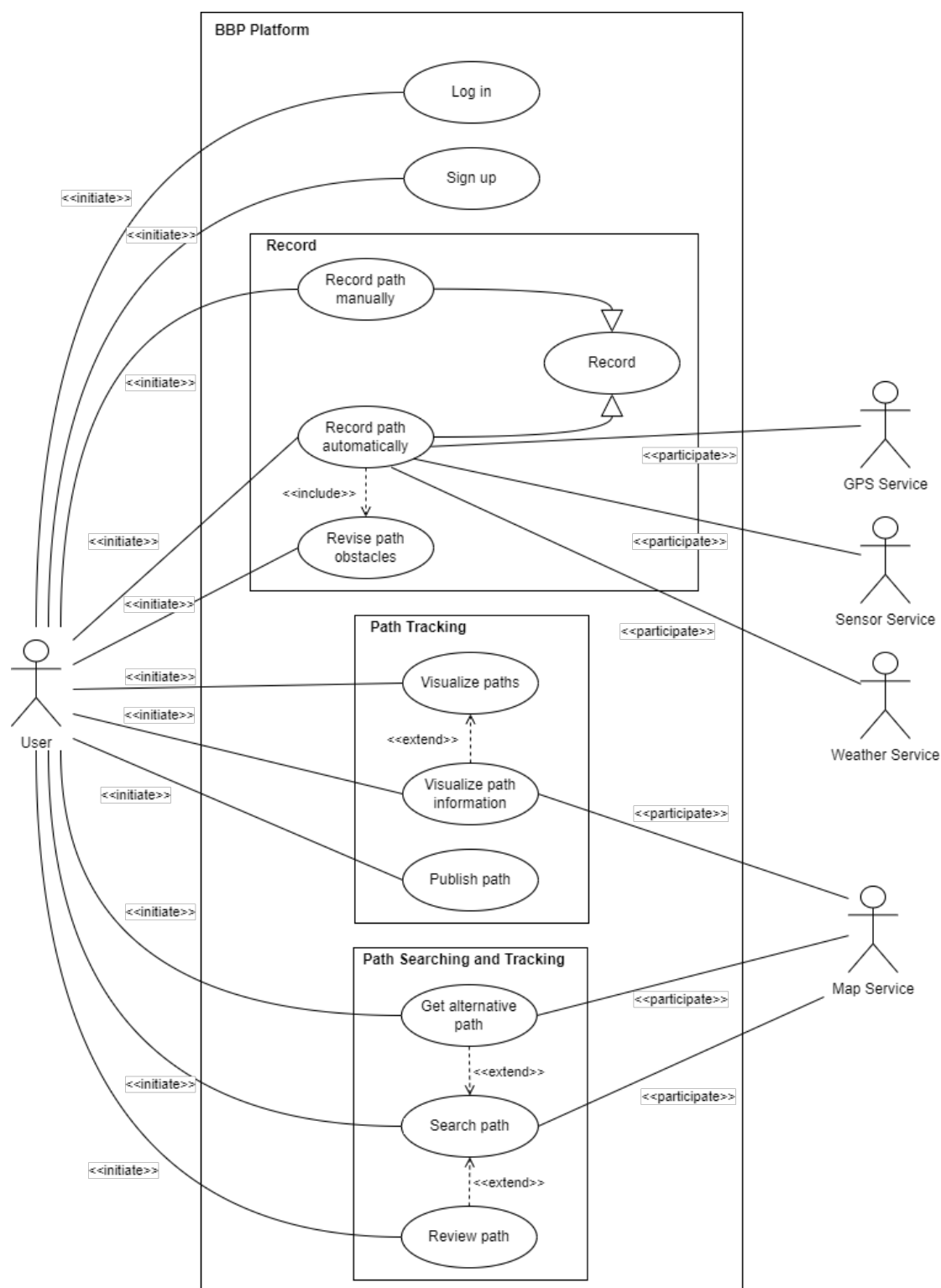


Figure 19: BBP Platform Use Case Diagram

The use-case diagram shown in Figure 19 presents the complete set of use cases that specify the functionality of the BBP platform, as well as the actors that trigger or participate in each use case. The actors include the user and several external systems:

- **GPS Service:** An external system responsible for obtaining and providing the real-time location of a user through a mobile device.
- **Sensor Service:** An external system responsible for collecting data detected by the accelerometer and gyroscope of different types of mobile devices.
- **Weather Service:** An external system responsible for providing real-time meteorological conditions for any location worldwide.
- **Map Service:** An external system that provides real-time route mapping based on specified locations and segments, with different markers along the path.

3.2.2 Use Cases

[UC1] Sign up

Name	Sign up
Actor	User
Entry Condition	True
Event Flow	1. On the homepage, the User clicks the "Log in" button. 2. On the log-in page, the User clicks the hyperlink with text "Don't you have an account yet?". 3. On the register page, the User inserts his name in the textfield labeled as "Name". 4. The User inserts the email, that will be the unique identifier of the account, in the textfield labeled as "Email". 5. The user inserts his password in the textfield labeled as password, and rewrites it the same in the textfield labeled as "Confirm Password". 6. The User clicks the "Register" button. 7. The BBP platform checks that the e-mail has not been used before and that the password satisfy the security constraints
Exit Condition	The use case ends when the user successfully creates an account using the data entered in the associated form.
Exception	The User email has already been taken, In this case, an error message is shown and the flow of events starts again from point 4.

[UC2] Log in

Name	Log in
Actor	User
Entry Condition	True
Event Flow	1. On the homepage, the User clicks the "Log in" button. 2. On the log-in page, the User inserts his email in the textfield labeled as "Email". 3. The User inserts his password in the textfield labeled as "Password". 4. The User clicks the "Log In" button. 5. The BBP platform displays the home page.
Exit Condition	The use case ends when the user successfully access the desired account and the home page is displayed.
Exception	Possible exceptions: 1. The inserted email does not correspond to any account 2. The password for the given email is incorrect In any of this cases, an error message is shown and the flow of events starts again from point 2.

[UC3] Record path manually

Name	Record path manually
Actor	User
Entry Condition	The User has an account in the BBP platform, is logged in and knows every street and the status about the bike path he wants to insert.
Event Flow	1. On the homepage, the User clicks the "Record" button located in the navbar, displaying the recording method window. 2. The User selects the "Manual" option, by clicking it in the bubble. 3. On the "Manual Record" page, the User selects the status of the bike path, by choosing it in a the dropdown Button. 4. The user inserts the name of the street in the bike path, by filling the text fields and adding new fields clicking the "Add" button. 5. The User clicks the "Confirm" button.
Exit Condition	The use case terminates when the bike path is successfully recorded in the BBP platform.
Exception	The User does not insert a path status. In this case, the manually record is not possible. Then, an error message is shown and the flow of events starts again from point 3. The User does not insert any street name. In this case, the manually record is not possible, since there is no street on the path and does not exist a path itself. Then, an error message is shown and the flow of events starts again from point 4. The user clicks the "Cancel recording" button. In this case, the user decides to discard this new record, then an error message is shown and the flow of events starts again from point 1.
Special Reqs	

[UC4] Record path automatically

Name	Record path automatically
Actor	User, GPS Service, Sensor service, Weather service
Entry Condition	The User has an account in the BBP platform, is logged in, has mobile device in their bike trip and enable the automatically path recording in the BBP platform.
Event Flow	1. On the homepage, the User clicks the "Record" button located in the navbar, displaying the recording method window. 2. The User selects the "Automatic" option, by clicking it in the bubble, displaying the "Automatic Record" page. 3. On the "Automatically Record" page, the User clicks the "Record" button. 4. The BBP platform records the bike path using the GPS Service. 5. The BBP platform recognize the obstacles in the path by requesting the information to the Sensor Service (Gyroscope and Accelerometer) of the users mobile device . 6. The BBP platform stores weather information by requesting the current conditions from the Weather service. 7. The User clicks the "Stop" button.
Exit Condition	The use case terminates when the BBP platform successfully record the bike path and their obstacles while the user is cycling.
Exception	On the "Automatic Record" page, there is another bike path that has already been automatically recorded, but its obstacles have not yet been confirmed or corrected. In this case, the user has first to revise automatic record. There are no Sensor or GPS services available on the mobile device running the BBP platform. In this case, an error message is shown and the flow of events starts again from point 2. There are no Weather service available on the mobile device running the BBP platform. In this case, the Users recording keeps going.
Special Reqs	

[UC5] Revise path obstacles

Name	Revise path obstacles
Actor	User
Entry Condition	The User has an account in the BBP platform, is logged in, has already automatically recorded a bike path and knows the obstacles encountered in the bike path.
Event Flow	1. On the homepage, the User clicks the "Record" button displayed in the navbar. 2. On the "Automatic Record" page, the User selects a street name (segment), where could be a wrongly detected obstacle, by clicking the "Obstacles" button with the pencil image. 3. On the "Edit Obstacle" page, the User either types new obstacles they found on the path into the text field and clicks the "Add" button, or clicks the X button to discard wrong detected obstacles. 4. The User clicks the "Confirm" button. 5. The BBP platform automatically displays "Automatic Record" page with the revised obstacles updated. 6. The User clicks the "Confirm" button.
Exit Condition	The use case ends when the user has successfully corrected or removed the incorrectly detected bike path obstacle, so that the old obstacle no longer appears and the new one is shown on the "Automatic Record" page.
Exception	The user cannot find any obstacle to correct. In this case, all recognized obstacles have been reviewed and the automatic recording is complete.
Special Reqs	

[UC6] Visualize paths

Name	Visualize paths
Actor	User
Entry Condition	The User has an account in the BBP platform and is logged in.
Event Flow	1. On the homepage, the User clicks the "My records" button displayed in the navbar. 2. On the "My Records" page, the BBP platform displays all paths recorded by the registered user. 3. The user clicks the "Manual" or "Automatic" button to view, respectively, the paths recorded manually or those recorded automatically with confirmed or corrected obstacles.
Exit Condition	The use case terminates when the user successfully visualize all the recorded paths associated with their account.
Exception	The user cannot see any path. In this case, the user has not recorded any bike path yet and can record a path by clicking the "Record a path" text button.
Special Reqs	

[UC7] Visualize path information

Name	Visualize path information
Actor	User, Map Service
Entry Condition	The user has an account on the BBP platform, is logged in, and has already recorded at least one bike path (automatically or manually).
Event Flow	1. On the homepage, the User clicks the "My records" button displayed in the navbar. 2. On the "My Records" page, the user selects the bike path to view from a list of recorded paths ordered by insertion date. 3. The BBP platform displays detailed information about the path's performance and condition, including statistics and, when available, weather data. 4. The BBP platform displays the different streets that the path crosses (segments), identified by their street names, along with the obstacles detected in each segment. 5. The Map service displays the recorded path on a map.
Exit Condition	The use case ends when the user has successfully viewed all information associated with the recorded path, including the detailed information of statistics, the segments and their obstacles, and the map representation.
Exception	On the "My Records" page, the User cannot visualize any path. In this case, the user haven't recorded any bike path yet and is able to record an path by clicking in the text button "Record a path". The user cannot visualize detailed information of statistics about this path. In this case, the path was recorded manually, so only street names and status data are available. The Map Service is not available. In this case, is just displayed the detailed information, the segments and their obstacles.
Special Reqs	

[UC8] Publish path

Name	Publish path
Actor	User
Entry Condition	The User has an account in the BBP platform, is logged in and has at least one recorded bike path.
Event Flow	1. On the homepage, the User clicks the "My records" button displayed in the navbar. 2. On the "My Records" page, the user selects the bike path to publish from a list of recorded paths ordered by insertion date. 3. The User clicks the "Publish" button. 4. The BBP platform displays a confirmation window notifying the user that the path will be visible to any user of the platform. 5. In the confirmation window, the User clicks "Confirm" button.
Exit Condition	The use case terminates when the user successfully makes their bike path public. The bike path is now available to appear in any search performed on the platform.
Exception	On the "My Records" page, the User cannot visualize any path. In this case, the user haven't recorded any bike path yet and is able to record an path by clicking in the text button "Record a path". In the confirmation window, the user clicks "Cancel" button. In this case, the User decides not to publish the recorded path. Then,the flow of events starts again from point 3.
Special Reqs	

[UC9] Search path

Name	Search path
Actor	User, Map Service
Entry Condition	True (an user can effect a search at any time)
Event Flow	1. On the homepage, the User inserts the origin point in the text field labeled as "Origin". 2. On the homepage, the User inserts the destination point in the text field labeled as "Destination". 3. The User clicks the "Search" button. 4. The Map Service displays the bike path on the map from the starting point to the destination, based on the segments that compose the path. 5. The BBP platform shows markers on the map indicating the obstacles along the path. 6. The BBP platform displays a status indicator for the path in one corner of the map.
Exit Condition	The use case terminates when the user successfully viewed the bike path from the specified origin to the specified destination.
Exception	The user is unable to view any bike path between the origin and destination points. In this case, the BBP platform has no recorded paths or segments (Associated to recorded paths) that start, end, or pass through those locations.
Special Reqs	

[UC10] Get alternative path

Name	Get alternative path
Actor	User, Map Service
Entry Condition	True (an user can effect a search at any time)
Event Flow	1. On the homepage, the User inserts the origin point in the text field labeled as "Origin". 2. On the homepage, the User inserts the destination point in the text field labeled as "Destination". 3. The User clicks the "Search" button. 4. The User clicks the "View more routes" button. 5. The BBP platform displays a list of different alternative paths, sorted according to their path score. 6. The User selects an alternative path by choosing it in a ordered list. 7. The Map Service displays the bike path on the map from the starting point to the destination, based on the segments that compose the path. 8. The BBP platform shows markers on the map indicating the obstacles along the path. 9. The BBP platform displays a status indicator for the path in one corner of the map.
Exit Condition	The use case terminates when the user successfully viewed alternatives bike paths from the specified origin to the specified destination.
Exception	The user is unable to view any bike path between the origin and destination points. In this case, the BBP platform has no recorded paths or segments (Associated to recorded paths) that start, end, or pass through those locations. The user cannot view any alternative bike path between the origin and destination points. In this case, the BBP platform lacks a sufficient number of recorded paths that begin, end, or pass through those locations to perform ordering by path score.
Special Reqs	The alternative list must have less than 5 bike paths.

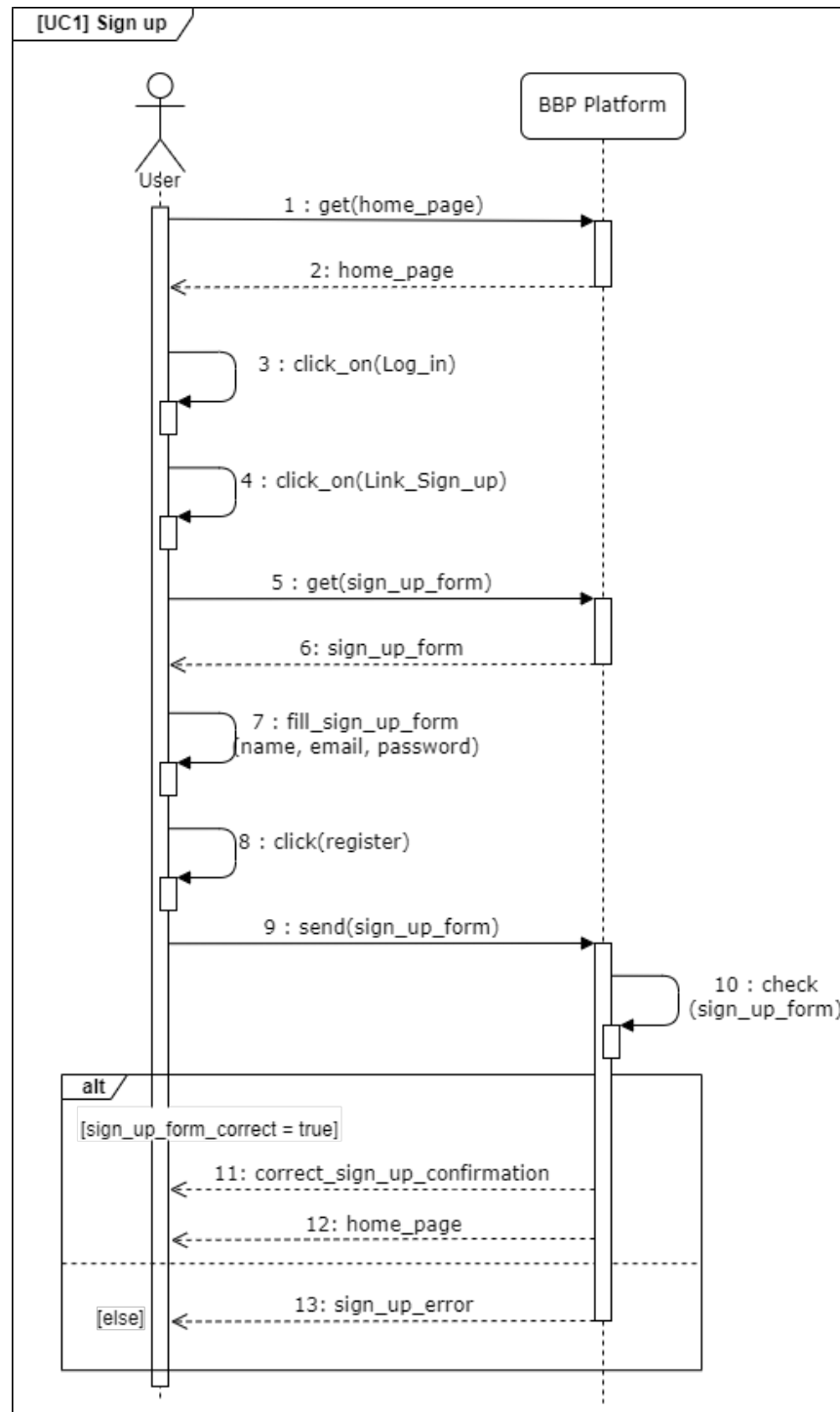
[UC11] Review path

Name	Review path
Actor	User
Entry Condition	The User has an account in the BBP platform and is logged in.
Event Flow	1. On the homepage, the User inserts the origin point in the text field labeled as "Origin". 2. On the homepage, the User inserts the destination point in the text field labeled as "Destination". 3. The User clicks the "Search" button. 4. The BBP platform displays the bike path on the map from the starting point to the destination. 5. The User clicks "Review" button. 6. On the "Review" page, the User selects the status of the bike path to review in the dropdown button. 7. On the "Review" page, for each segment where they found an obstacle, the user clicks the "+" button located to the left of the street name associated with that segment. 8. On the "Review obstacles" page for a segment, the user types the obstacle they found and clicks the "Add" button. 9. After adding the obstacles, the user clicks the "Accept" button. 10. The User clicks "Submit Review" button.
Exit Condition	The use case ends when the bike path review has been successfully submitted to the BBP platform. The review is now available to the BBP platform and the path score of the selected path and its segments is updated.
Exception	The user is unable to view any bike path between the origin and destination points. In this case, the BBP platform has no recorded paths or segments (Associated to recorded paths) that start, end, or pass through those locations. The User does not insert a path status. In this case, the path review is not possible, since the Status field is required. Then, an error message is shown and the flow of events starts again from point 6. 37 The User clicks the "Cancel Review". In this case, the User decides to discard their path review, then the flow of events starts again from point 5.

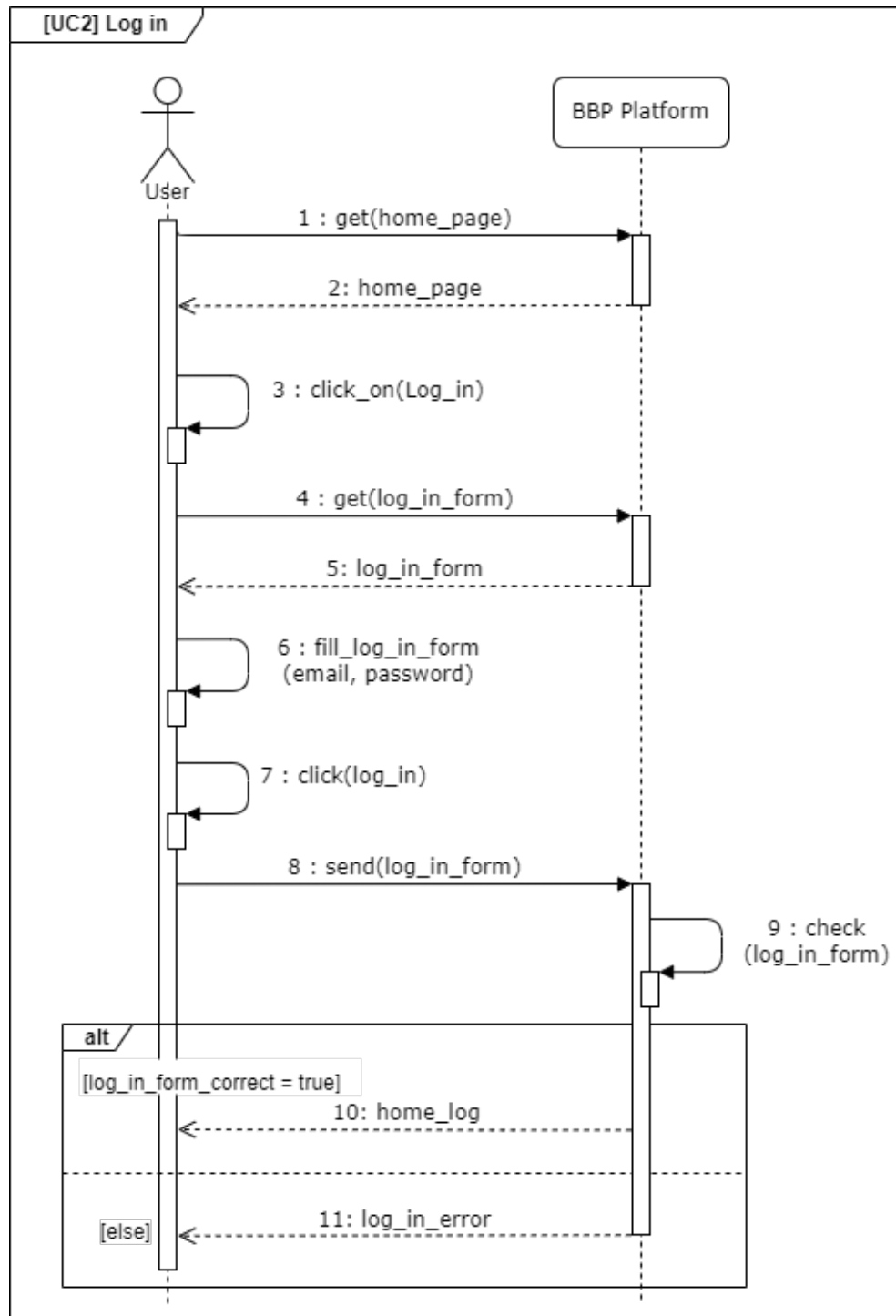
3.2.3 Sequence Diagrams

Please note that the notifications here are modeled at a high level, without considering design and architectural aspects, and omitting synchronization details. They have been introduced only to represent the interaction with users.

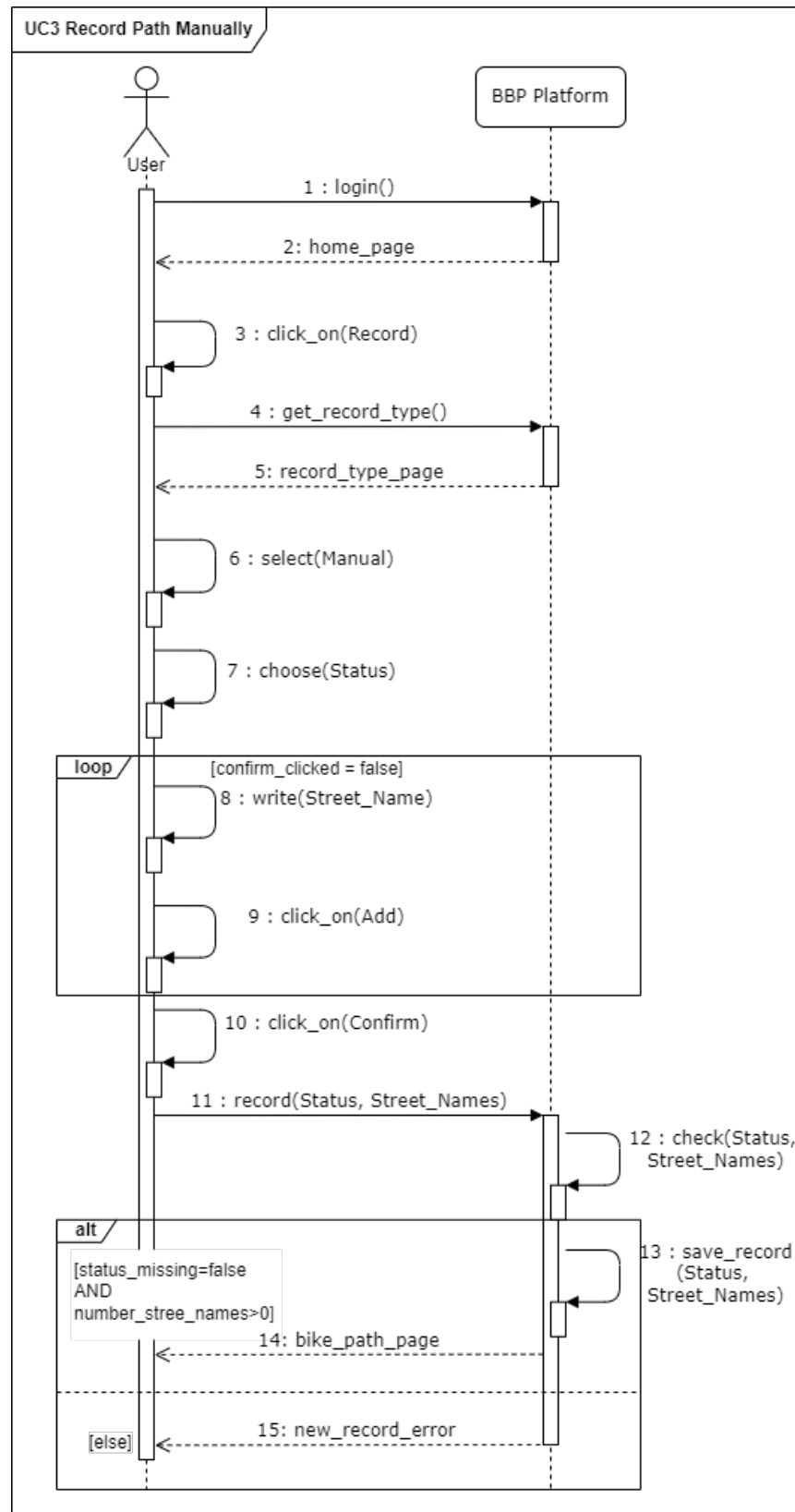
[UC1] Sign up



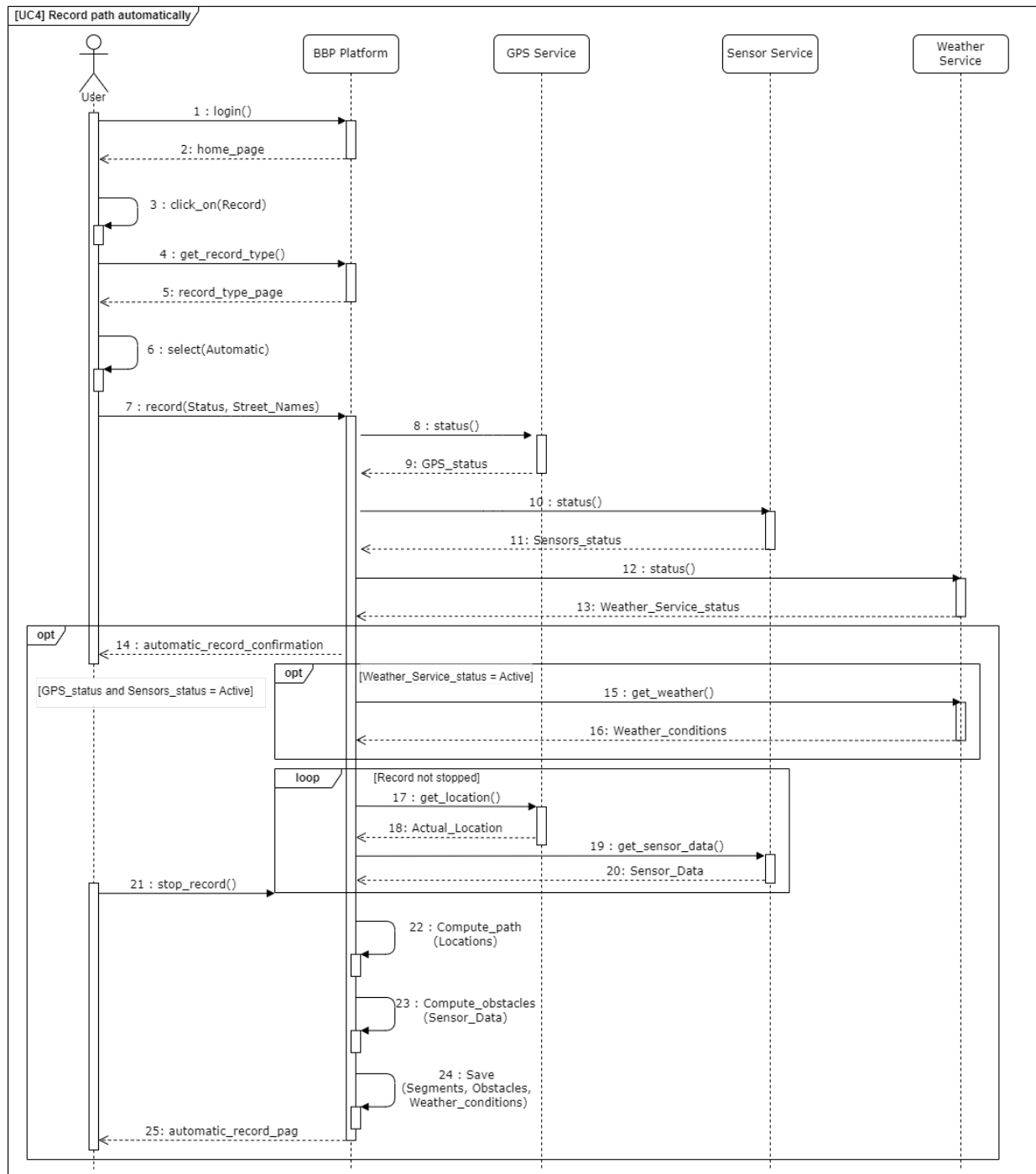
[[UC2] Log in



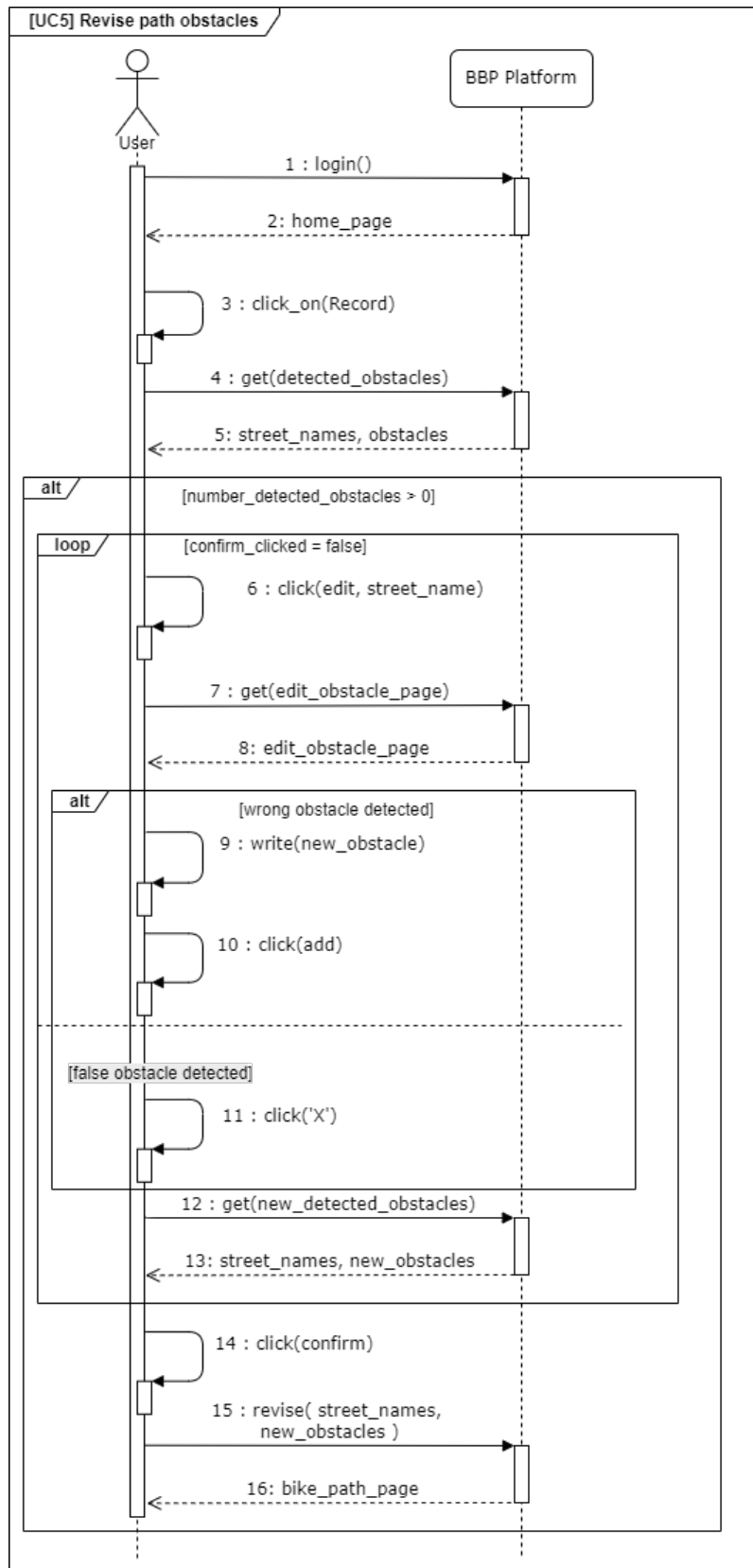
[UC3] Record path manually



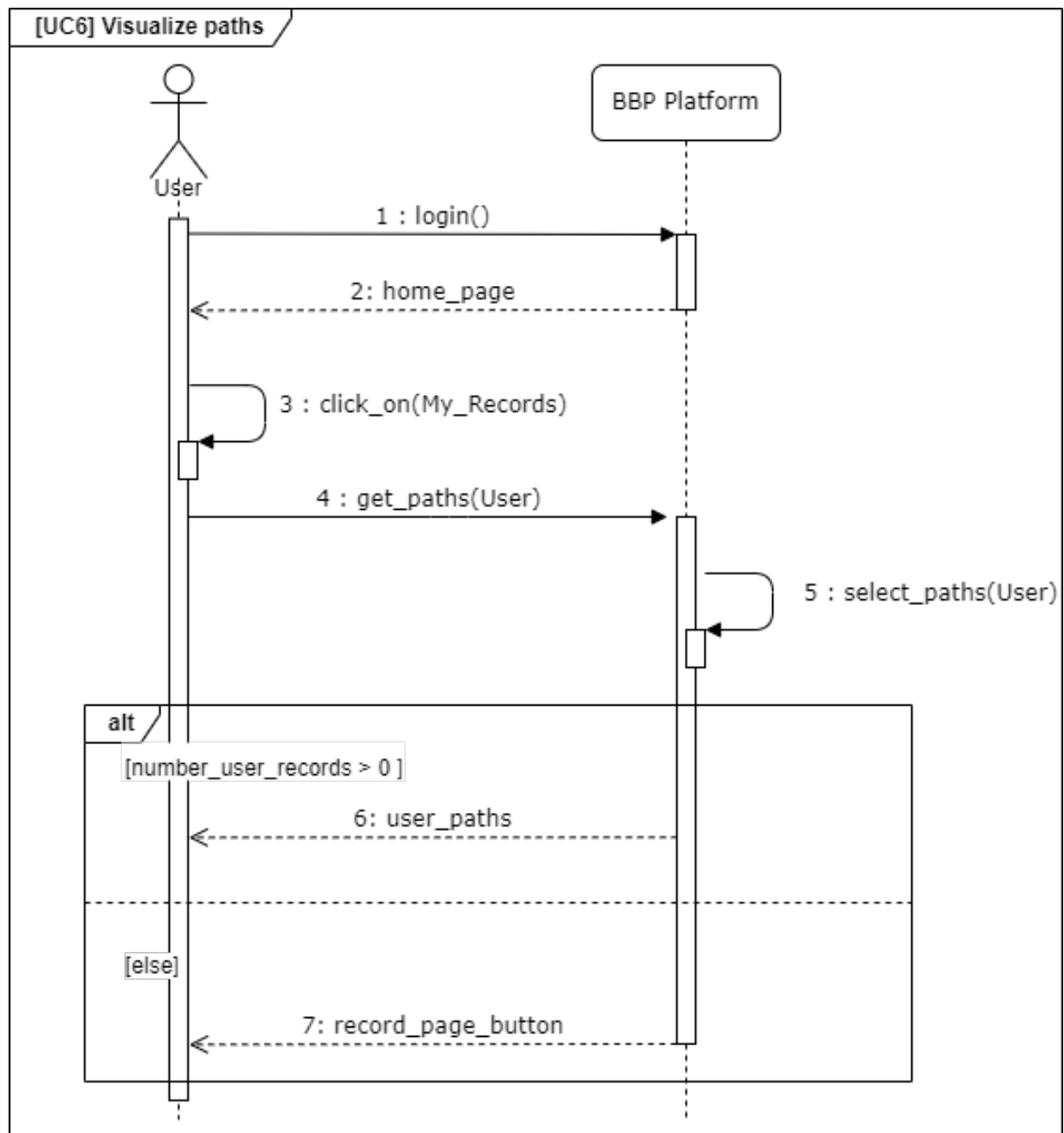
[UC4] Record path automatically



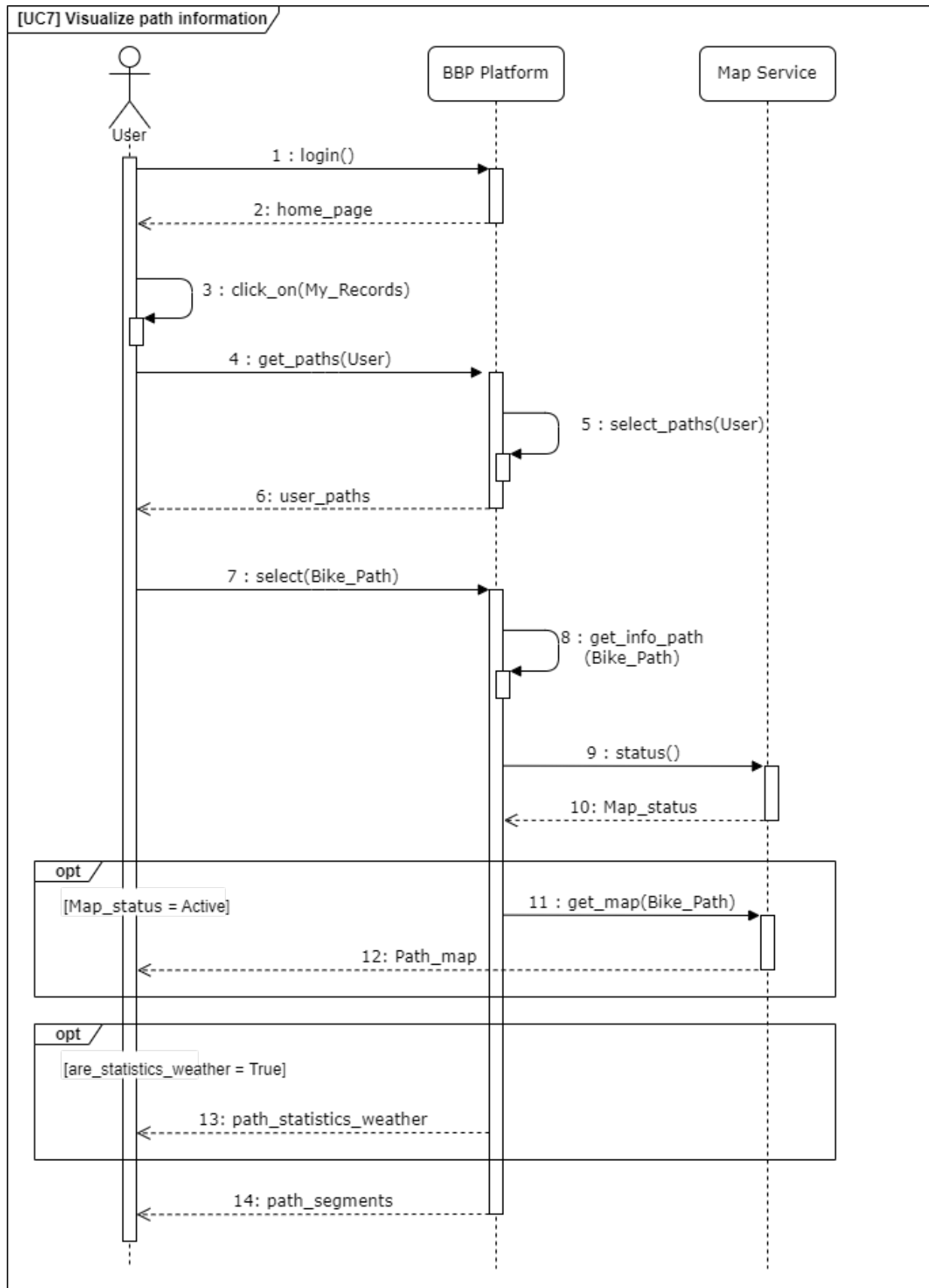
[UC5] Revise path obstacles



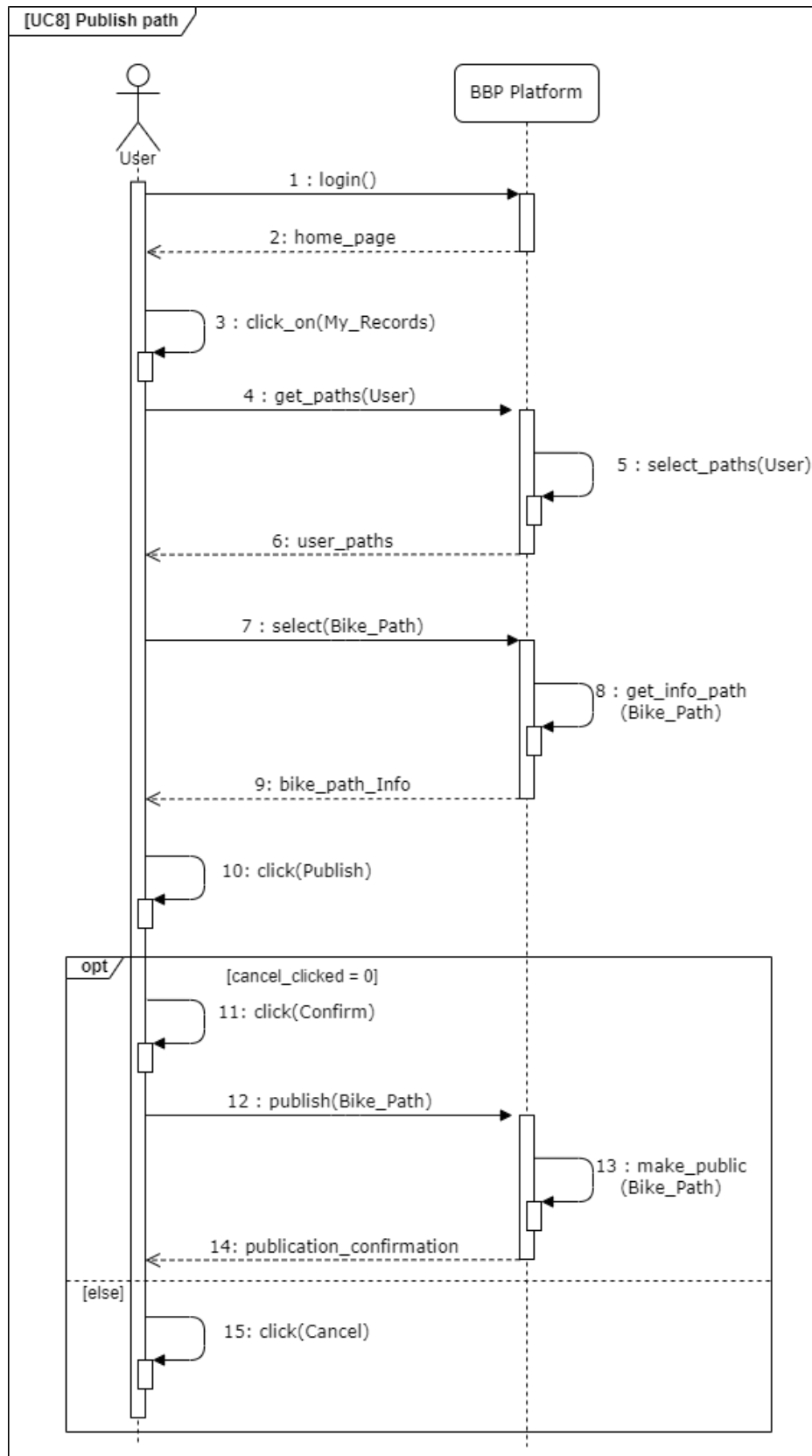
[UC6] Visualize paths



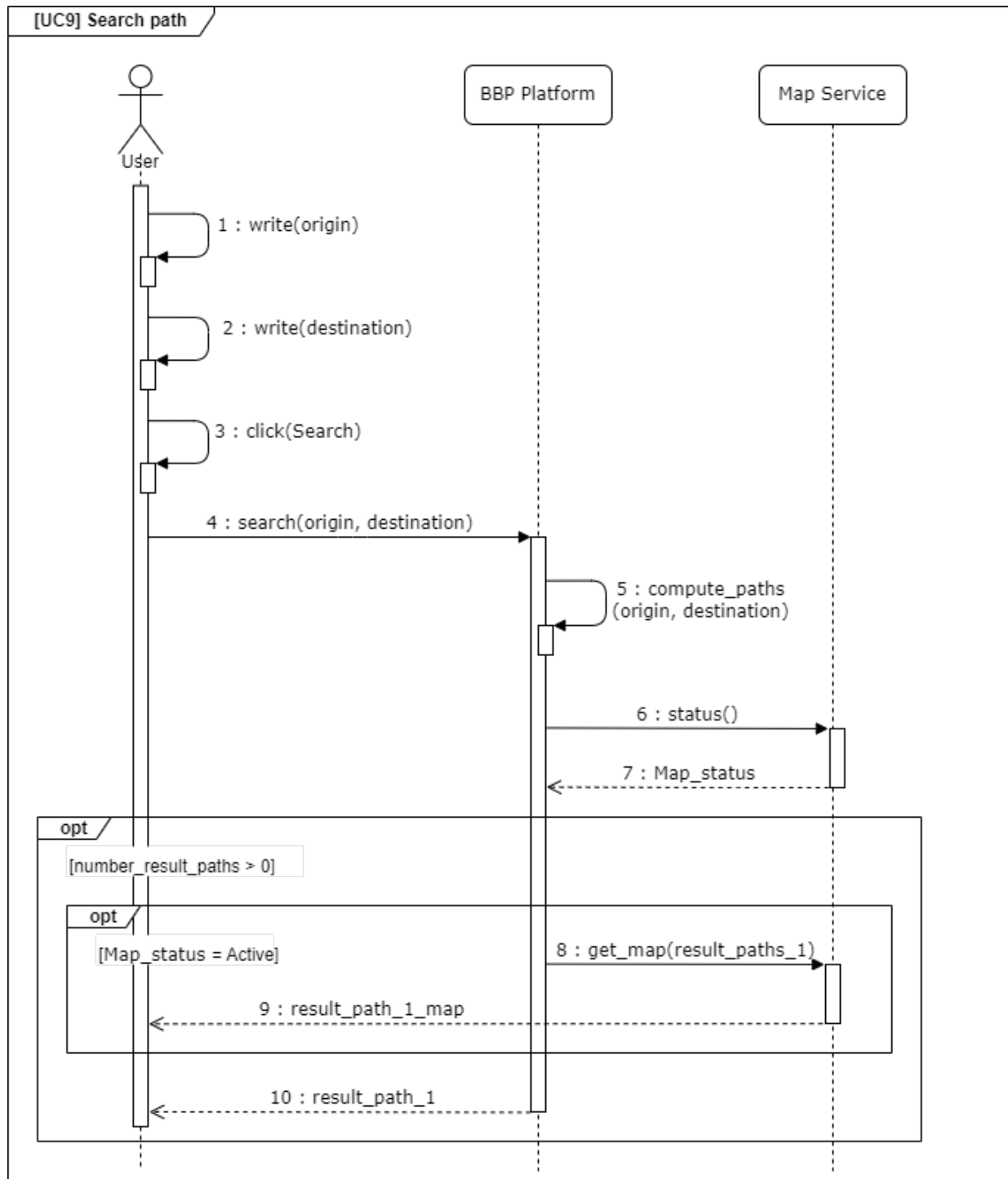
[UC7] Visualize path information



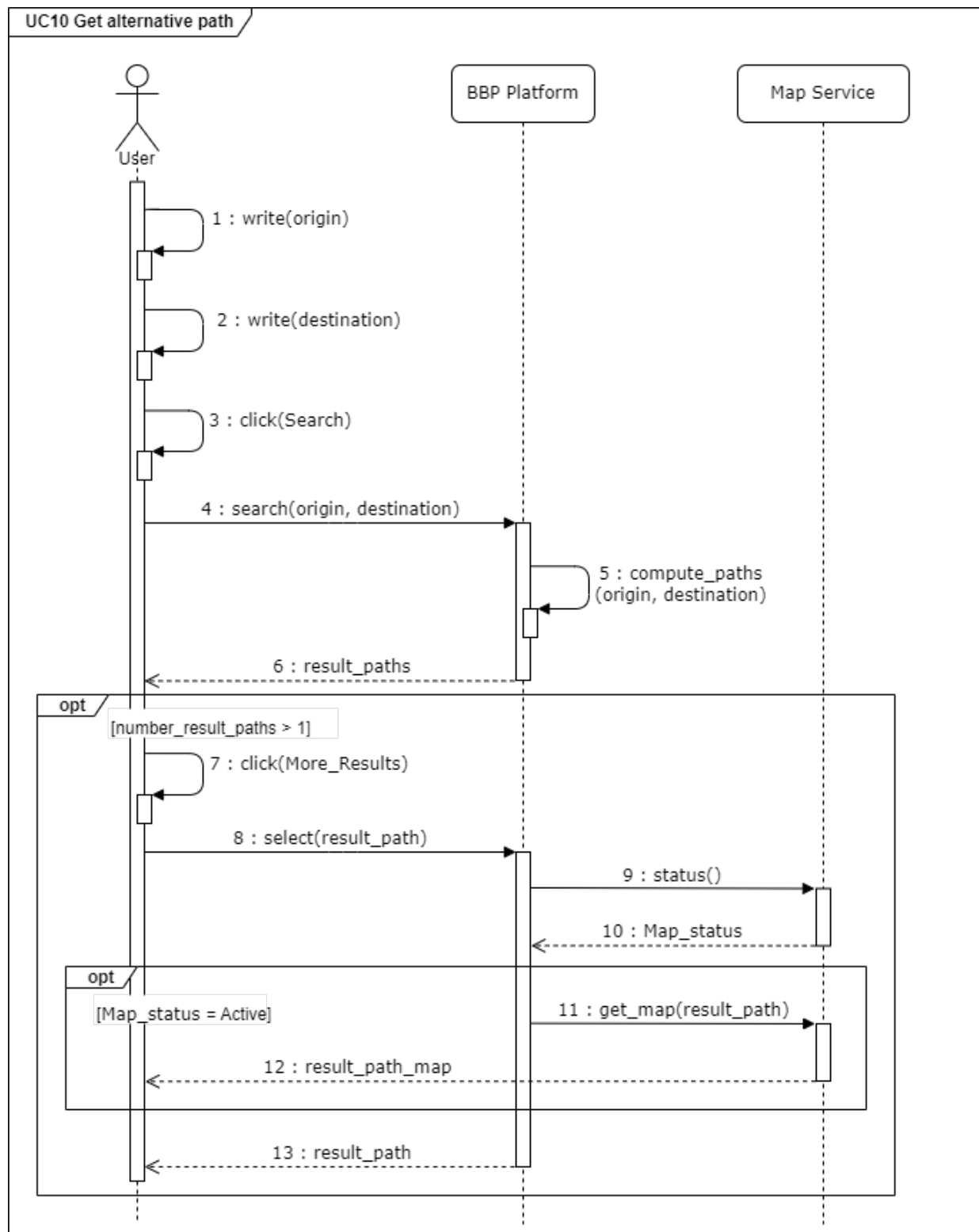
[UC8] Publish path



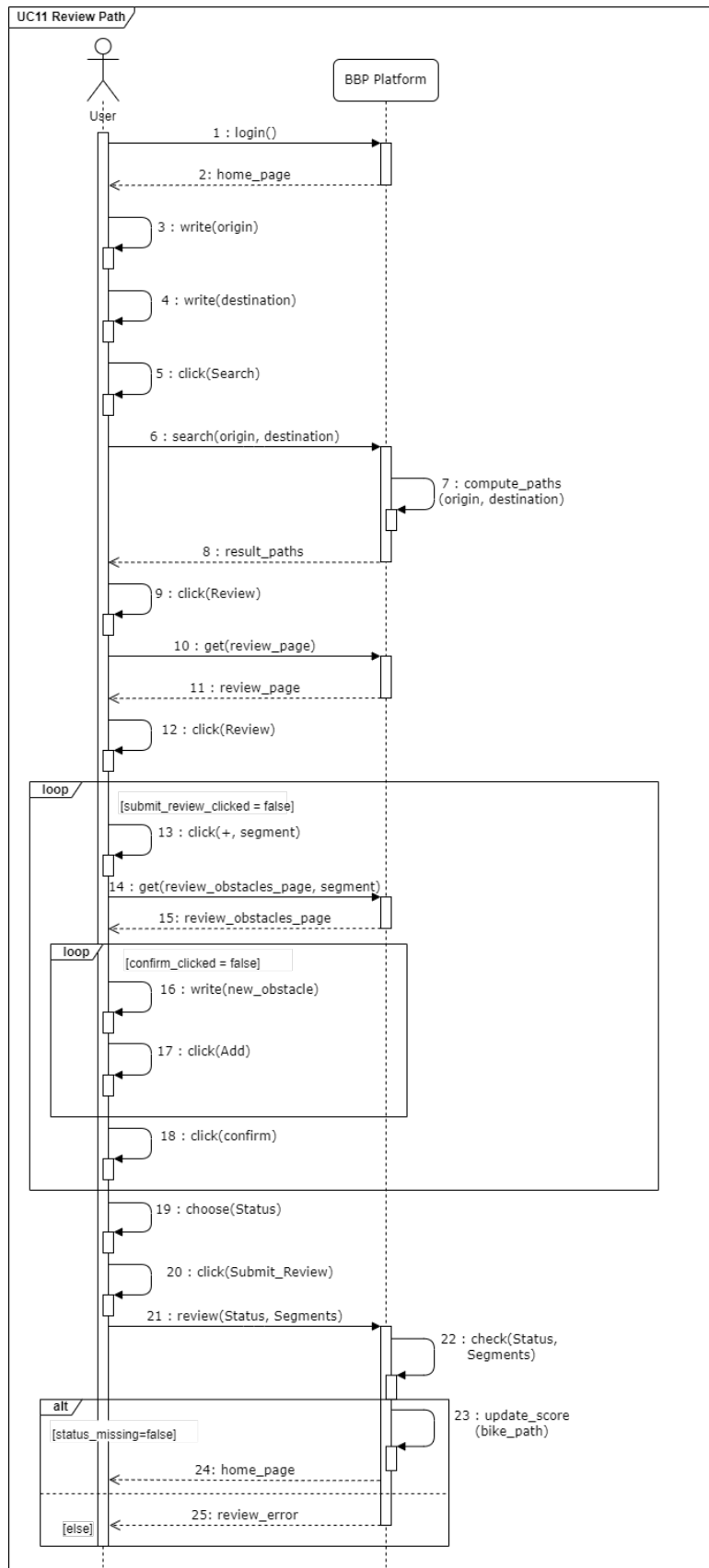
[UC9] Search path



[UC10] Get alternative path



[UC11] Review path



3.2.4 Requirement Mapping

G1: Cyclists would be able to record the bike routes they use, along with any obstacles they encounter.	
• R1: When an unregistered user enters the BBP platform, it shall allow the user to create an account. • R2: When a registered user enters the BBP platform, it shall allow them to log into their account. • R3: The BBP platform shall allow a logged-in user to manually record a bike path by providing the names of the streets in the bike path and their status. • R4: When a logged-in user enters the BBP platform, it shall allow them to automatically record a bike path they are taking. • R5: When a logged-in user automatically records a bike path, the BBP platform shall recognize obstacles in the path. • R6: When a logged-in user with automatically recorded bike paths enters the BBP platform, it shall allow them to confirm the presence of obstacles automatically recognized in the recorded process. • R7: When a logged-in user with automatically recorded bike paths enters the BBP platform, it shall allow them to correct automatically recognized obstacles in the path.	• D1: Users report the bike path status truthfully. • D2: Users report obstacles observed on the bike path truthfully. • D3: The User's mobile device has a fully operative GPS. • D4: The User's mobile device has a fully operative accelerometer and gyroscope sensors. • D5: The User does not use a "Fake GPS" while recording a path. • D7: Each segment of the bike path taken by the user have traceable address. • D8: Bike paths are where a proper bike path exists. • D9: Bike paths are where cars are rare and speed limits are compatible with the average speed of a bike. • D10: All users have a reliable internet connection.

We show that the requirements set $R_1 = \{R1, R2, R3, R4, R5, R6, R7\}$ and the domain assumptions set $D_1 = \{D1, D2, D3, D4, D5, D7, D8, D9, D10\}$ guarantee G1:

- From R1, initially, no User has an account on the BBP platform and therefore cannot perform any recording action.

- From R1, R2 and D10, each user can create and access an account in the BBP platform.

First, we show that a User is able to record a previously taken bike route in manual mode

- From D7, D8 and D9, the User has taken a valid bike route that can be recorded and traceable on a map.
- From R3, D1 and D7, the User is able to correctly record the bike path taken by providing the identified addresses and the overall status observed.

Second, we show that a User is able to record a bike route they are currently taking.

- From D7, D8 and D9, the User is taking a valid bike route that can be recorded and traceable on a map.
- From R4, D3, D5 and D7, the BBP platform is able to correctly record the bike the User is taking.
- From R5 and D4, the BBP platform also recognizes obstacles associated with changes in sensor data.
- From R6, R7, D2 and D4, the User confirms or modifies the obstacles detected by the BBP platform and submits the correctly observed obstacles in the bike path taken.

Finally, it is clear that both cases show that R_1 ensures satisfaction of G1 in the context of domain assumptions D_1 .

G2: Cyclists would be able to keep track of the bike paths they take.

• R1: When an unregistered user enters the BBP platform, it shall allow the user to create an account. • R2: When a registered user enters the BBP platform, it shall allow them to log into their account. • R3: The BBP platform shall allow a logged-in user to manually record a bike path by providing the names of the streets in the bike path and their status. • R4: When a logged-in user enters the BBP platform, it shall allow them to automatically record a bike path they are taking. • R8: The BBP platform shall show to a logged-in user their list of recorded paths.	• D1: Users report the bike path status truthfully. • D3: The User's mobile device has a fully operative GPS. • D5: The User does not use a "Fake GPS" while recording a path. • D7: Each segment of the bike path taken by the user have traceable address. • D8: Bike paths are where a proper bike path exists. • D9: Bike paths are where cars are rare and speed limits are compatible with the average speed of a bike. • D10: All users have a reliable internet connection.
---	---

We show that the requirements set $R_2 = \{R1, R2, R3, R4, R8\}$ and the domain assumptions set $D_2 = \{D1, D3, D5, D7, D8, D9, D10\}$ guarantee G2:

- From R1, initially, no User has an account on the BBP platform and therefore cannot perform any recording action.
- From R1, R2 and D10, each user can create and access an account in the BBP platform.
- From D7, D8 and D9, the User has taken a valid bike route that can be recorded and traceable on a map.
- From R3, R4, D1, D3, D5 and D7, the User correctly records a bike path in either of two ways:
 1. Manual recording: the user enters the identified streets and reports the overall observed status.
 2. Automatic recording: the system records the bike path using GPS data from the user's mobile device.
- From R8, the User is able to track the bike paths the user has taken.

Finally, R_2 ensures the satisfaction of G2 in the context of domain assumptions D_2 .

G3: Cyclists would be able to view detailed information and statistics for the bike routes they have taken.

- | | |
|---|--|
| <ul style="list-style-type: none"> • R1: When an unregistered user enters the BBP platform, it shall allow the user to create an account. • R2: When a registered user enters the BBP platform, it shall allow them to log into their account. • R3: The BBP platform shall allow a logged-in user to manually record a bike path by providing the names of the streets in the bike path and their status. • R4: When a logged-in user enters the BBP platform, it shall allow them to automatically record a bike path they are taking. • R9: The BBP platform shall provide a logged-in user the path statistics about their automatically recorded paths. • R10: When the Weather service is available, the BBP platform shall provide weather information to a logged-in user for their automatically recorded paths. • R13: When a logged-in user selects a previously recorded bike path, the BBP platform shall display a map view showing the selected bike path and its associated obstacles. • R14: When a logged-in user selects a previously automatically recorded bike path, the BBP platform shall display the path statistics computed in the recording process for the selected bike path. | <ul style="list-style-type: none"> • D1: Users report the bike path status truthfully. • D3: The User's mobile device has a fully operative GPS. • D5: The User does not use a "Fake GPS" while recording a path. • D7: Each segment of the bike path taken by the user have traceable address. • D8: Bike paths are where a proper bike path exists. • D9: Bike paths are where cars are rare and speed limits are compatible with the average speed of a bike. • D10: All users have a reliable internet connection.. |
|---|--|

• R15: When a logged-in user selects a previously automatically recorded bike path that has weather information, the BBP platform shall display the stored weather information for the selected bike path. • R16: When a logged-in user selects a previously recorded bike path, the BBP platform shall display the list of streets traversed by the selected bike path in the stored segment order. • R17: When a logged-in user selects a previously recorded bike path, the BBP platform shall display the obstacles associated with the selected bike path showing for each obstacle its type and the street where it was detected.	
---	--

We show that the requirements set $R_3 = \{R1, R2, R3, R4, R9, R10, R13, R14, R15, R16, R17\}$ and the domain assumptions set $D_3 = \{D1, D2, D3, D5, D7, D8, D9, D10\}$ guarantee G3:

- From R1, initially, no User has an account on the BBP platform and therefore cannot perform any recording action.
- From R1, R2 and D10, each user can create and access an account in the BBP platform.
- From D7, D8 and D9, the User has taken a valid bike route that can be recorded and traceable on a map.

First, we show that a User is able to record a previously taken bike route in manual mode

- From R3, D1 and D7, the User is able to correctly record the bike path taken by providing the identified addresses and the overall status observed.

Second, we show that a User is able to record a bike route they are currently taking.

- From R4, D3, D5 and D7, the BBP platform is able to correctly record the bike the User is taking.
- From R9, R10, D3 and D5, the BBP platform collects the performance statistics and the weather information of the path bike the User has taken.

Finally, from R13, R14, R15, R16 and R17, the User visualizes all the information available of a bike path taken by the user. Then, we showed that R_3 ensures satisfaction of G3 in the context of domain assumptions D_3 .

G4: Cyclists would be able to publish their bike paths and their reviews of other bike paths so that other cyclists can see and use them.	
• R1: When an unregistered user enters the BBP platform, it shall allow the user to create an account. • R2: When a registered user enters the BBP platform, it shall allow them to log into their account. • R3: The BBP platform shall allow a logged-in user to manually record a bike path by providing the names of the streets in the bike path and their status. • R4: When a logged-in user enters the BBP platform, it shall allow them to automatically record a bike path they are taking. • R8: The BBP platform shall show to a logged-in user their list of recorded paths. • R11: When a logged-in user with recorded paths enters the BBP platform, it shall allow them to make their bike paths visible to other users who are searching for routes. • R12: The BBP platform shall allow a logged-in user to insert new status and obstacles information to a public bike path. • R18: When a user enters the BPP platform, it shall allow the user to search bike paths considering an origin and a destination point.	• D1: Users report the bike path status truthfully. • D2: Users report obstacles observed on the bike path truthfully. • D3: The User's mobile device has a fully operative GPS. • D5: The User does not use a "Fake GPS" while recording a path. • D6: Path's origin and destination have traceable addresses. • D7: Each segment of the bike path taken by the user have traceable address. • D8: Bike paths are where a proper bike path exists. • D9: Bike paths are where cars are rare and speed limits are compatible with the average speed of a bike. • D10: All users have a reliable internet connection.

We show that the requirements set $R_4 = \{R1, R2, R3, R4, R8, R11, R12, R18\}$ and

the domain assumptions set $D_4 = \{D1, D2, D3, D5, D7, D8, D9, D10\}$ guarantee G4:

- From R1, initially, no User has an account on the BBP platform and therefore cannot perform any recording action.
- From R1, R2 and D10, each user can create and access an account in the BBP platform.
- From D7, D8 and D9, the User has taken a valid bike route that can be recorded and traceable on a map.
- From R3, R4, D1, D3, D5 and D7, the User correctly records a bike path in either of two ways:
 1. Manual recording: the user enters the identified streets and reports the overall observed status.
 2. Automatic recording: the system records the bike path using GPS data from the user's mobile device.
- From R8 and R11, the User is able to track the bike paths the user has taken and, if they choose, publish them so that they appear in search results.
- From R18, D6, and D7, the User can enter an origin and destination point to search for a bike path and retrieve public paths shared by other users.
- From R12, D1, and D2, the BBP platform allows the User to submit a review of a public bike path, including the path's status and any obstacles observed while riding.

Finally, R_4 ensures the satisfaction of G4 in the context of domain assumptions D_4 .

G5: Cyclists would like to be able to search for optimal and well-maintained bike paths by specifying an origin and a destination.

• R18: When a user enters the BPP platform, it shall allow the user to search bike paths considering an origin and a destination point. • R19: When a user performs a bike-path search and the BBP platform does not have recorded bike paths that satisfy the search parameters, the BBP platform shall generate candidate bike paths by combining recorded segments (including their associated obstacles) from existing bike paths. • R20: The BBP platform shall display bike paths options (First one on a map), corresponding to an user search (with the origin and destination point) in descending order depending on their path score.	• D6: Path's origin and destination have traceable addresses. • D7: Each segment of the bike path taken by the user have traceable address. • D8: Bike paths are where a proper bike path exists. • D9: Bike paths are where cars are rare and speed limits are compatible with the average speed of a bike. • D10: All users have a reliable internet connection.
--	--

We show that the requirements set $R_5 = \{R18, R19, R20\}$ and the domain assumptions set $D_5 = \{D6, D7, D8, D9, D10\}$ guarantee G5:

- From D7, D8, and D9, Users who record a bike path must follow a valid route that can be recorded and traced on a map.
- From R18, R19, D6, and D7, any User can enter an origin and destination point to search for a bike path and retrieve either public paths shared by other users or paths generated by the BBP platform from existing path segments.
- From R20, D6, and D7, the user visualizes the suggested bike path on a map and other options based on their associated path scores.

Finally, R_5 ensures the satisfaction of G5 in the context of domain assumptions D_5 .

G6: Cyclists would like the most up to date information regarding a bike path according to other cyclists' experiences.

- | | |
|---|--|
| <ul style="list-style-type: none"> • R1: When an unregistered user enters the BBP platform, it shall allow the user to create an account. • R2: When a registered user enters the BBP platform, it shall allow them to log into their account. • R3: The BBP platform shall allow a logged-in user to manually record a bike path by providing the names of the streets in the bike path and their status. • R4: When a logged-in user enters the BBP platform, it shall allow them to automatically record a bike path they are taking. • R11: When a logged-in user with recorded paths enters the BBP platform, it shall allow them to make their bike paths visible to other users who are searching for routes. • R12: The BBP platform shall allow a logged-in user to insert new status and obstacles information to a public bike path. • R19: When a user performs a bike-path search and the BBP platform does not have recorded bike paths that satisfy the search parameters, the BBP platform shall generate candidate bike paths by combining recorded segments (including their associated obstacles) from existing bike paths. | <ul style="list-style-type: none"> • D1: Users report the bike path status truthfully. • D2: Users report obstacles observed on the bike path truthfully. • D3: The User's mobile device has a fully operative GPS. • D5: The User does not use a "Fake GPS" while recording a path. • D6: Path's origin and destination have traceable addresses. • D7: Each segment of the bike path taken by the user have traceable address. • D8: Bike paths are where a proper bike path exists. • D9: Bike paths are where cars are rare and speed limits are compatible with the average speed of a bike. • D10: All users have a reliable internet connection. |
|---|--|

• R21: When the BBP platform receives new status and obstacle information from a public bike path, it shall update their path score according to all updates made to the path information, prioritizing the most up-to-date information • R22: When the BBP platform generates new bike paths in response to a user bike-path search, the BBP platform shall estimate and assign an overall status to each generated bike path, deriving that status from the statuses of the existing bike paths that include the segments used to form it.	
---	--

We show that the requirements set $R_6 = \{R1, R2, R3, R4, R11, R12, R19, R21, R22\}$ and the domain assumptions set $D_6 = \{D1, D2, D3, D5, D6, D7, D8, D9, D10\}$ guarantee G6:

- From R1, initially, no User has an account on the BBP platform and therefore cannot perform any recording action.
- From R1, R2 and D10, a user can create and access an account in the BBP platform.
- From D7, D8 and D9, a logged-in User has taken a valid bike route that can be recorded and traceable on a map.
- From R3, R4, D1, D3, D5 and D7, a logged-in User correctly records a bike path in either of two ways:
 1. Manual recording: the user enters the identified streets and reports the overall observed status.
 2. Automatic recording: the system records the bike path using GPS data from the user's mobile device.
- From R11, a logged-in User is able to **publish** the bike paths the user has taken so that they appear in search results.
- From R12, D1, and D2, the BBP platform allows a logged-in User to submit a **review** of a public bike path, including the path's status and any obstacles observed while riding.
- From R19, R22, D6, and D7, the BBP platform can **generate new bike paths** from stored public paths and assign each one a specific status.

- From R21, the BBP Platform computes an overall path score to provide the user with the most up-to-date information, using the status of the **public paths**, the **reviews**, and the computed status of the **generated paths**.

Finally, R_6 ensures the satisfaction of G6 in the context of domain assumptions D_6 .

3.3 Non-functional requirements

3.3.1 Security

- The BBP Platform must implement an authorization system that controls access to user accounts and assigns privileges for the actions of recording, storing, publishing, and reviewing paths, ensuring that only authorized users can perform these actions.
- The BBP Platform must implement an authentication system that verifies user identity through email and password, ensuring secure and accurate user authentication.
- The BBP Platform must require the user to register with a password that follows the guidelines for password strength exposed in the NIST standard.
- All communication between the BBP platform's components and external services must be encrypted using HTTPS.

3.3.2 Compatibility

- The BBP platform must support Android mobile devices.
- The BBP platform must be compatible with the motion sensors (Accelerometer and gyroscope) in Android devices.

3.3.3 Scalability

- The BBP platform must be horizontally scalable, adding additional server instances until hardware limit is reached, enabling the platform to handle a growing number of users.
- The BBP platform must support a scalable load balancing mechanism that distribute user traffic evenly across multiple server instances.

3.3.4 Performance

- The BBP Platform must accurately record the user's location with a margin of error not exceeding 500 meters in real-time during a trip.

3.3.5 Availability

- The BBP Platform must continue to record the bike path (in automatic mode) even when the mobile device temporarily loses internet connectivity.

3.3.6 Interoperability

- The BBP platform must support internal and external systems communication via a defined API.

3.4 Design Constraints

3.4.1 Standards compliance

- GDPR: The system manages users' personal and administrative data, thus it must be compliant with the General Data Protection Regulation according to the European guidelines.
- ISO/IEC 27001: The system must be characterized by a structured and comprehensive information security management, in order to protect sensitive information and be adaptable to new risks.

3.4.2 Hardware limitations

The BBP platform is a web application optimized for mobile devices; therefore, hardware limitations are primarily related to the capabilities of smartphones, which are required to support the system's full functionality. While the application can be accessed through a standard web browser on other devices, some features may be unavailable outside a mobile environment:

- Full functionality of the system requires a mobile device equipped with integrated GPS and motion sensors, including an accelerometer and a gyroscope. These hardware components are necessary to support automatic path recording and obstacle detection.
- The platform can also be accessed through desktop or laptop browsers; however, devices lacking mobile sensors are limited to features such as path browsing, searching, and manual path insertion.
- Interaction with the platform depends on the availability of a stable internet connection. A minimum bandwidth of 5 Mbps is required to ensure timely page loading and efficient data transmission.

3.4.3 Any other constraint

The browser used to access the system must be up to date and compliant with modern web standards. JavaScript and cookies must be enabled to ensure correct execution of the application's core functionalities.

3.5 Software System Attributes

3.5.1 Reliability

The BBP platform shall ensure reliable operation and data integrity by storing user-generated data transactionally and replicating it across multiple nodes to prevent loss and maintain consistency. It shall also use automated backups and a tested disaster-recovery process to restore route history and public-path data after failures or accidental deletion. If external services (map/GPS, weather) are unavailable, BBP shall keep core features working (e.g., trip recording with later synchronization) and notify users when optional features are temporarily missing.

3.5.2 Availability

The BBP platform shall provide a highly available service for route search, record access, and publishing/reviewing, targeting at least 99.9% uptime (excluding scheduled maintenance) through redundancy and automatic failover of critical components. The system shall handle predictable demand peaks such as weekday commuting windows and weekend daytime usage using horizontal scaling and load balancing so responsiveness remains stable under traffic spikes. To limit outages caused by slow/unavailable external dependencies, the platform shall apply timeouts and graceful degradation (e.g., showing routes without weather data), and schedule maintenance in low-usage windows (typically late-night on weekdays).

3.5.3 Security

The BBP platform stores sensitive user data (e.g., identity and credentials) and processes personal data such as location and trip metrics, so it shall enforce strong protection for storage and communications. All communications shall use HTTPS/TLS, sensitive data at rest shall be encrypted, and passwords shall never be stored in plaintext but only as salted, slow hashes to limit the impact of data leaks. The system shall mitigate common web threats like injection and denial-of-service (DoS) through strict input validation, parameterized queries, rate limiting, and protective gateway rules. In addition, the platform shall apply privacy-by-design measures required by GDPR, such as collecting only data necessary for the service, restricting access to personal data, and supporting user control over their data (e.g., export/deletion) with defined retention policies.

3.5.4 Maintainability

The BBP platform shall be easy to maintain and evolve by adopting a modular architecture that separates core concerns (authentication, path recording/search, reviews, and external map/GPS and weather integrations). The codebase shall follow consistent standards and common design patterns, supported by clear documentation (architecture, deployment, and API specifications). To limit regressions, the system shall rely on automated tests (unit/integration/end-to-end) and a CI pipeline that runs tests and static checks on every change before release.

3.5.5 Portability

The BBP platform shall be a web-based application to ensure portability across major devices and operating systems, delivering a consistent experience on modern browsers. The UI shall be responsive and optimized for smartphones and tablets, adapting to different screen sizes and input methods. Since some features depend on mobile hardware (GPS, accelerometer, gyroscope), BBP shall use progressive enhancement: core functions remain available everywhere, while automatic recording is enabled only when the required sensors and permissions are present.

4 FORMAL ANALYSIS USING ALLOY

The formal analysis using Alloy aims to formally prove the correctness of the model described in all the previous sections. To achieve this, the following goals have been tested through different Alloy features (assertions and predicates mainly):

- **G1 - Cyclists would be able to record the bike routes they use, along with any obstacles they encounter :** The predicate *makeTrip* models the recording of a new trip performed by a *User*. When *makeTrip* is executed, the system moves to a new state where a bike path has been recorded together with the obstacles encountered and the detected status information. Moreover, the predicate *ShowNewRecord* shows an instance of the model where a new trip is recorded.
- **G2 and G3 - Cyclists would be able to keep track of the bike paths they take and to view detailed information and statistics for the bike routes they have taken :** To support these goals, the assertion *NoTripWithoutUser* checks that every recorded trip (including its detected status and obstacles) is always associated with exactly one *User*. This formally captures the fact that users can keep track of their own trips. Moreover, the predicate *ShowUserTrips* characterizes the operation of viewing the trips (together with detected status and obstacles) that a user has made on a given bike path.
- **G4 - Cyclists would be able to publish their bike paths and their reviews of other bike paths so that other cyclists can see and use them :** The predicate *publishTrip* models the publication of a previously recorded trip by a *User*. When *publishTrip* is executed, the system transitions to a new state where that trip (including detected obstacles and status) becomes public and therefore visible to other cyclists, which reflects directly on the status of the bike path. In addition, the predicate *ShowPublishRecord* shows an instance of the model where a recorded trip is published by an user.
- **G5 - Cyclists would like to be able to search for optimal and well-maintained bike paths by specifying an origin and a destination :** To satisfy this goal, the assertion *NoBikePathWithoutStatusAndSegments* checks that every bike path is defined with an estimated status and its segments. These elements are necessary to

support searches based on an origin and a destination (i.e., the initial and final points of the path).

- **G6 - Cyclists would like the most up to date information regarding a bike path according to other cyclists' experiences :** This goal is addressed by linking bike-path updates to the publication of new trips. In particular, the predicate *publishTrip*, followed by a call to the predicate *updatePath*, models the action in which a user publishes a new trip on a specific bike path. As a consequence, the bike path information (status and obstacles) is updated based on that cyclist's experience. In this model, the update is simplified by assigning the most recent trip status to the bike path status, and aggregating the obstacles reported across published trips to preserve the historical obstacle information. Additionally, the predicate *ShowChangeStatusBikePaths* exhibits an instance of the model behavior when multiple updates exists.

At the same time, the *Predicates/Functions* and *System Simulation* section includes a simulation of the world's behavior, obtained by combining operations for recording (*MakeTrip* predicate), publishing (*PublishTrip* predicate) and updating (*updatePath* predicate) different trips and bike paths.

4.1 Alloy Code

// SIGNATURES (Class mapping)

```
sig User {
    var makes : set Trip
}

sig BikePath{
    var status: one PathStatus,
    var contains: set Trip,
    isComposedOf: some PathSegment
}

sig Trip{
    has: one PerformanceMetrics,
    detects: set Obstacle,
    status: one PathStatus,
    var currentState: one State
}

sig PathSegment{
    var contains: set Obstacle,
    initialPoint: one Position,
    endPoint: one Position
}
```



```

sig Obstacle{}

sig PerformanceMetrics{
    weather: set WeatherCondition
}

enum PathStatus{Optimal, Medium, Sufficient, RequiresMaintenance}
enum WeatherCondition{Sunny, Cloudy, Rainy, Windy, Stormy, Snowy}
enum State{Inactive, Active, Public}

sig Position{}

// FACTS

// A bike path does not have the same path segments with other bike path
fact {
    all path1: BikePath, path2: BikePath |
        always ((path1 != path2 and
            path1.isComposedOf = path2.isComposedOf)
            implies path1.isComposedOf = none)
}

// A trip performed in a path is always in that path
fact {
    all path: BikePath |
        always ( path.contains in path.contains' )
}

// The status of a BikePath doesn't change unless a trip on it is
// published
fact {
    all path: BikePath |
        always (
            (path.status' = path.status) implies
                (all trip: path.contains |
                    (trip.currentState != Inactive) implies
                        (trip.currentState = trip.currentState'))
            else(
                (one trip: path.contains |
                    trip.currentState = Active and
                    trip.currentState' = Public and
                    trip.status = path.status'
                )
            )
        )
}

```

```

}

// Any ACTIVE trip is performed JUST on one bikePath
fact{
    all trip: Trip |
        always ( (trip.currentState != Inactive) implies (
            one path: BikePath |
                trip in path.contains) )
}
// Any INACTIVE trip is not related with any bike path
fact{
    all path: BikePath, trip: Trip |
        always ( (trip in path.contains) implies
            (trip.currentState != Inactive) )
}
// Any ACTIVE trip is made by exactly 1 User
fact {
    all trip: Trip |
        always ( (trip.currentState != Inactive) implies (
            one user: User |
                trip in user.makes) )
}
// Any trip has its own performMetrics
fact{
    all metric: PerformanceMetrics |
        always (one trip: Trip |
            metric in trip.has)
}
// Any INACTIVE trip has not yet been made by an user
fact{
    all trip: Trip |
        always ( (trip.currentState = Inactive)
            implies (trip not in User.makes) )
}
//Current state of a trip should remain logically
fact{
    all trip:Trip | {
        always( trip.currentState = Inactive
            implies
                historically trip.currentState = Inactive )
        always( trip.currentState = Inactive
            implies
                eventually trip.currentState = Active )
        always( trip.currentState = Active

```

```

        implies
        after always ( trip.currentState = Active
or trip.currentState = Public) )
always( trip.currentState = Public
        implies
        (once trip.currentState = Active) and
        (after always trip.currentState = Public) )
    }
}

//Any path segment is part of a bike path
fact{
    all segment: PathSegment |
        always (segment in BikePath.isComposedOf)
}

// A obstacle reported on a segment is always in that segment
fact {
    all segment: BikePath |
        always ( segment.contains in segment.contains' )
}

//Any segment cannot have same intial and end position
fact{
    all segment: PathSegment |
        always (segment.initialPoint != segment.endPoint)
}

//The location segments of a bike path should be continuous
fact{
    all path: BikePath |
        always (#path.isComposedOf = 1
            or
            all segment1: path.isComposedOf |
                ((one segment2:path.isComposedOf |
                    segment1!=segment2 and
                    segment1.endPoint = segment2.initialPoint)
                or
                (one segment2:path.isComposedOf |
                    segment1!=segment2 and
                    segment1.initialPoint = segment2.endPoint)
            )
        )
}

```

```

//Any obstacle must be on JUST one bike path segment
fact{
    all o: Obstacle |
        always( (one segment: PathSegment |
            o in segment.contains)
            or
            o not in PathSegment.contains)
}
//Any obstacle must be reported in JUST one trip
fact{
    all o: Obstacle |
        always ( one trip: Trip |
            o in trip.detects )
}
// A trip report an obstacle in a segment of the Bike Path where the ACTIVE
//trip was taken
fact {
    all o: Obstacle, trip: Trip |
        always ( (o in trip.detects and trip.currentState != Inactive)
        implies (
            one path: BikePath |
                trip in path.contains and
                o in path.isComposedOf.contains
            ) )
}
// Any obstacle reported on an INACTIVE trip is not in a path segment
fact {
    all o: Obstacle, trip: Trip|
        always ( (trip.currentState = Inactive and o in trip.detects)
        implies (o not in BikePath.isComposedOf.contains) )
}

// A user made a trip forever
fact{
    all user: User |
        always (user.makes in user.makes')
}

// PREDICATES/FUNCTIONS

// A user records a new bike path he has taken
pred makeTrip[ user: User, path: BikePath]{
    one trip: Trip | {
        trip.currentState = Inactive

```

```

    // Trip made on a bike path
    path.contains' = path.contains + trip
    // Trip made by the user
    user.makes' = user.makes + trip
    // Obstacles detected in the trip on the path segments
    all obstacle: trip.detects |
        one segment: path.isComposedOf |
            segment.contains' = segment.contains + obstacle
    //Mark trip as recorded
    trip.currentState' = Active
    all t: Trip - trip | t.currentState' = t.currentState
}

}

// A user publish a bike path he has already recorded
pred publishTrip[ user: User ]{
    one trip: user.makes | {
        trip.currentState = Active

        //Mark trip as public
        all t: Trip - trip | t.currentState' = t.currentState
        trip.currentState' = Public
        //Next step perform update

        one path: BikePath |
            (trip in path.contains)
        implies (after updatePath[trip, path])
    }
}

// The bike path status is updated
pred updatePath[trip: Trip , path: BikePath ]{
    (path.status != trip.status) implies (
        path.status' = trip.status
    )
}

// Helper function: One step without doing nothing
pred doNothing{
    User' = User
    BikePath' = BikePath
    Trip' = Trip
    PathSegment' = PathSegment
    Obstacle' = Obstacle
}

```

```

        PerformanceMetrics' = PerformanceMetrics
        Position' = Position
    }

    //Helper functions: get active and inactive trips
    fun InactiveTrips : set Trip { {t: Trip | t.currentState = Inactive} }
    fun ActiveOnlyTrips : set Trip { {t: Trip | t.currentState = Active} }

    // One status change per step
    fact OneTripStateChangePerStep {
        always lone t: Trip | t.currentState' != t.currentState
    }

    // GOALS VERIFICATION

    // G1
    pred ShowNewRecord [t: Trip ] {
        t.currentState = Inactive
        t.currentState' = Active
        #t.detects = 1
    }

    // G2 - G3
    assert NoTripWithoutUser {
        all trip: Trip - InactiveTrips |
            always( one user: User |
                trip in user.makes )
    }
    pred ShowUserTrips [u: User]{
        #u.makes >2
        #u.makes.detects=3
    }

    //G4
    pred ShowPublishRecord [t: Trip ] {
        t.currentState = Active
        t.currentState' = Public
        #t.detects = 1
    }

```

```

//G5
assert NoBikePathWithoutStatusAndSegments{
    all path: BikePath |
        always(path.status != none and path.isComposedOf!=none and path.isCor
path.isComposedOf.endPoint!=none)
}

```

```

//G6
pred ShowChangeStatusBikePaths [u: User] {
    one t: u.makes |
        {
            one path: BikePath | {
                t in path.contains
                path.status != t.status
            }
            t.currentState = Active
            t.currentState' = Public
            #t.detects = 1
        }
    some t1:Trip | {
        t1.currentState= Inactive
    }
    some t1:Trip | {
        t1.currentState= Active
    }
    #BikePath = 2
    #User = 2
}

```

```

// RUNS
check NoTripWithoutUser for 15 but 3 steps
run ShowNewRecord for 3 but exactly 2 steps
run ShowUserTrips for 4 but exactly 2 steps
run ShowPublishRecord for 3 but exactly 2 steps
check NoBikePathWithoutStatusAndSegments for 15 but 3 steps
run ShowChangeStatusBikePaths for 5 but exactly 15 steps

```

// SYSTEM SIMULATION

```

//Initialization
//All paths private (Recorded or not yet recorded)
fact{
    all trip: Trip |

```

```

        trip.currentState = Inactive or trip.currentState = Active
    }
    //Some paths recorded and not recorded
    fact InitDiverse {
        (some t: Trip | t.currentState = Inactive)
        and
        (some t: Trip | t.currentState = Active)
    }

    fact systemBehavior{
        #ActiveOnlyTrips = 2
        #User = 2
        always (
            (some InactiveTrips or some ActiveOnlyTrips)
            implies
            (
                ( one user: User, path:BikePath | makeTrip[ user , path ])
                or
                (one user: User | publishTrip[ user ])
            )
        ) else
        (some InactiveTrips)
        implies
        (
            ( one user: User, path:BikePath | makeTrip[ user , path ])
        ) else
        (some ActiveOnlyTrips)
        implies
        (
            (one user: User | publishTrip[ user ])
        ) else
        doNothing
    )
}

```

4.2 Scenarios

- While executing the predicate *ShowNewRecord*, the modeled world includes three trips—two recorded and one yet to be recorded. Executing the recording action establishes a new user–trip link and a bike path–trip relationship, which in turn links the trip to its obstacles.

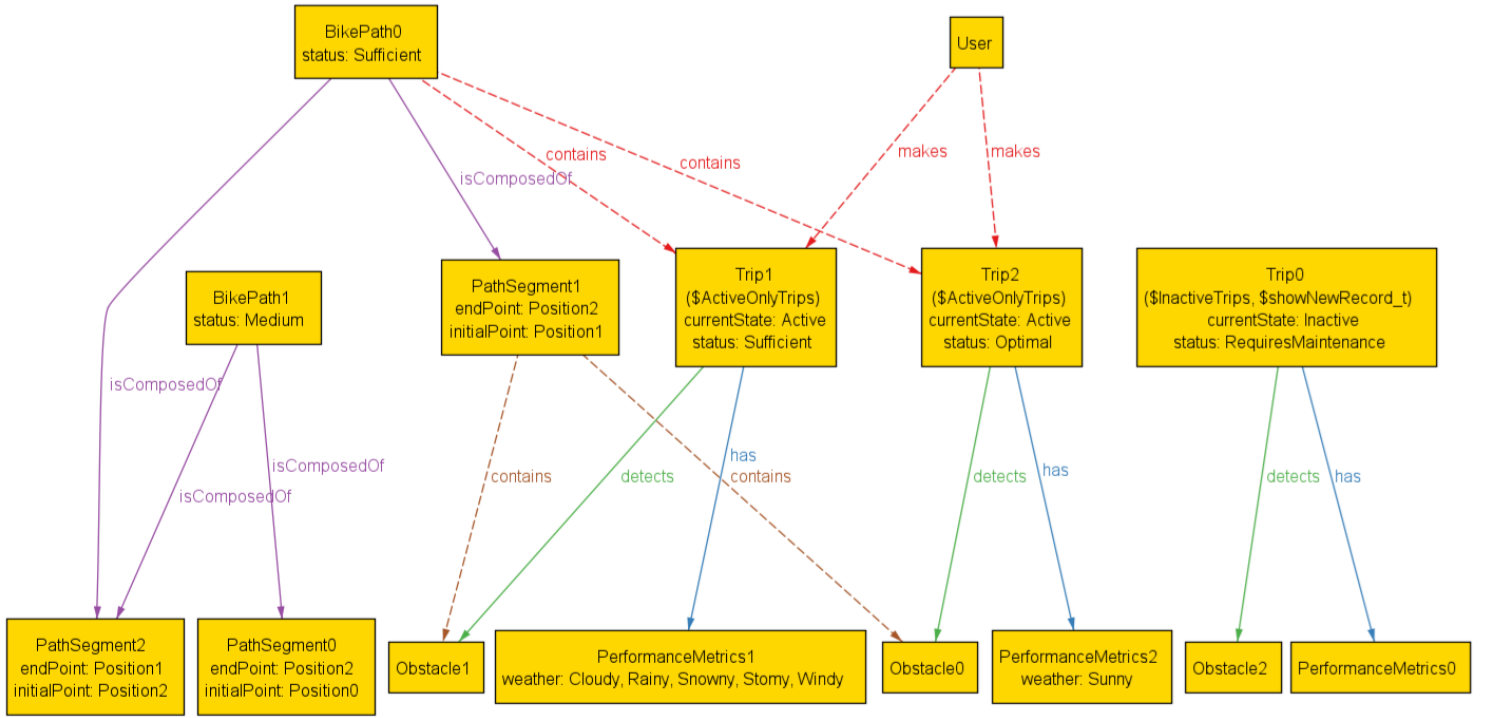


Figure 20: First step execution *ShowNewRecord*

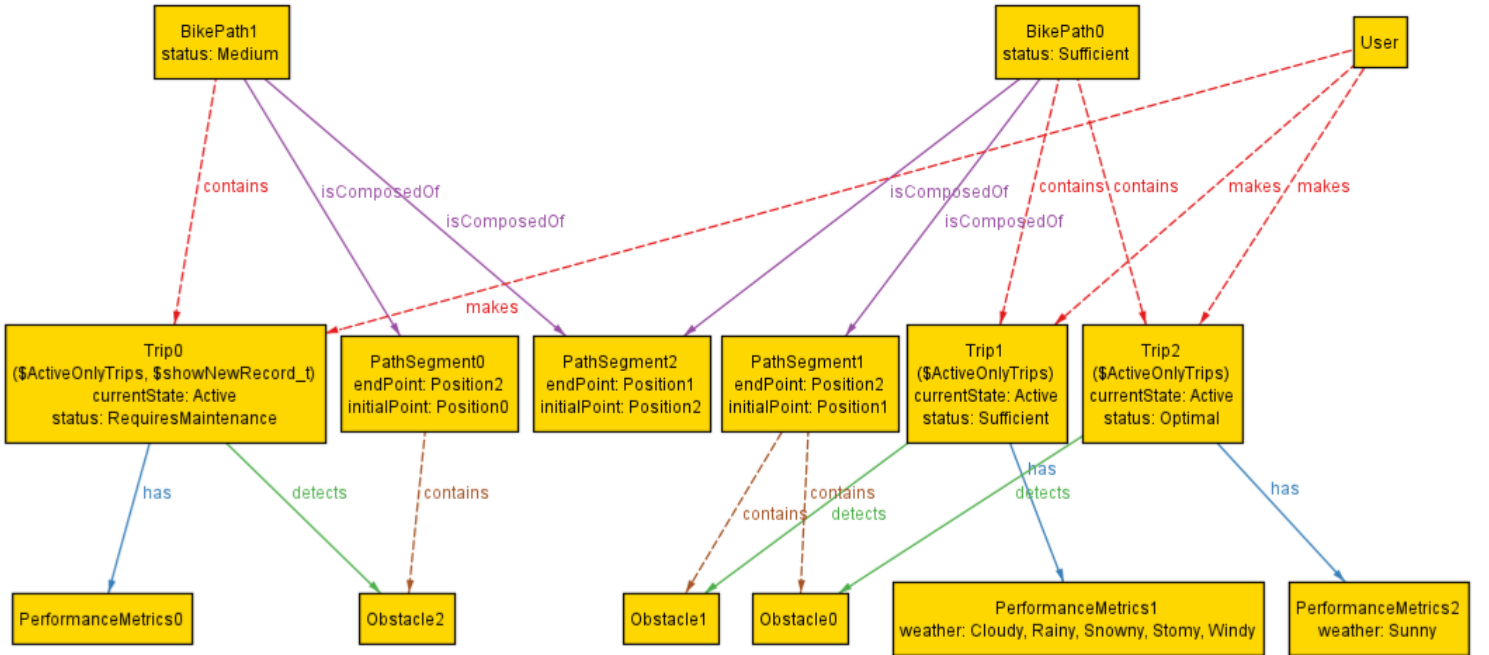


Figure 21: Second step execution *ShowNewRecord*

- It is evident that, during the execution of the predicate *ShowUserTrip*, the modeled world makes it possible to identify the trips made by each user, clearly distinguishing among all trips and their respective users.

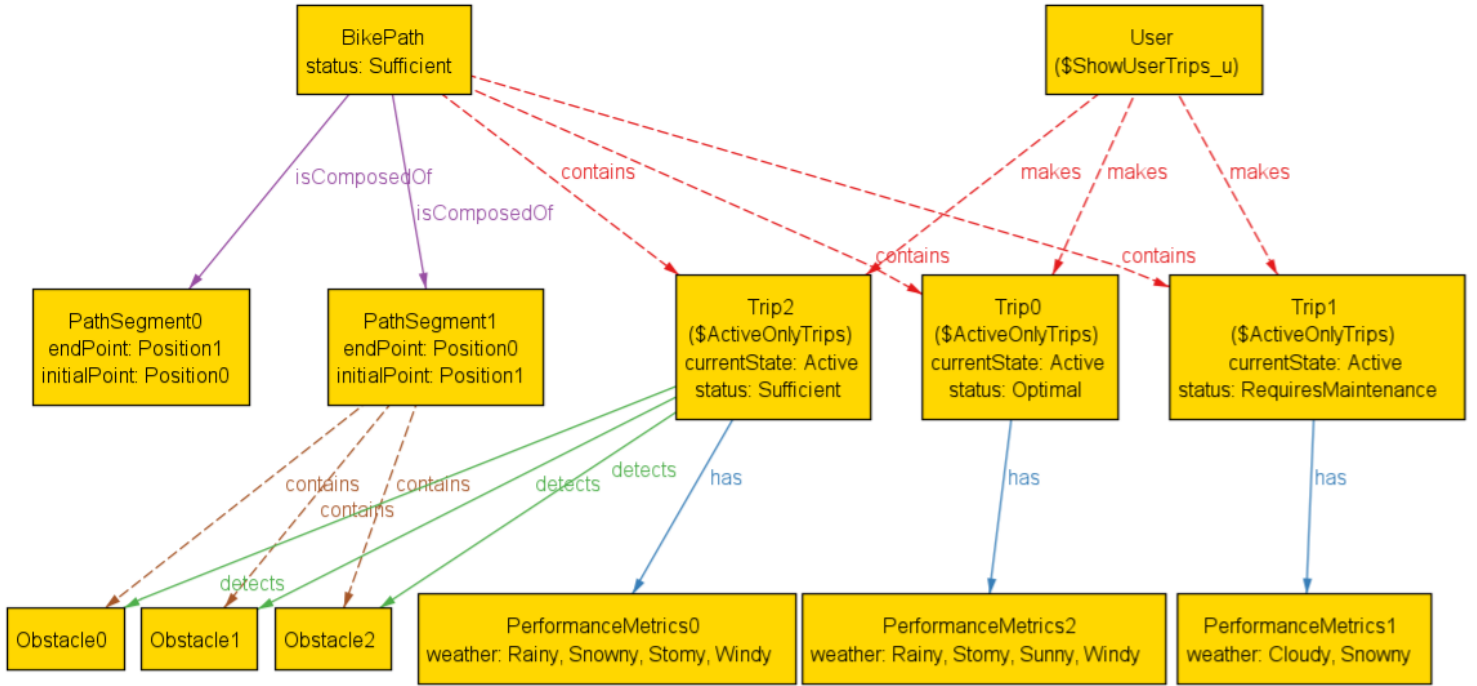


Figure 22: Model instance execution *ShowUserTrip*

- The execution of the predicate *ShowPublishRecord* starts from a model in which a user has three trips recorded on two different paths (Figure 23). As shown in Figure 24, *trip2* becomes public; consequently, the bike path on which it was performed is updated to reflect the newly public trip.

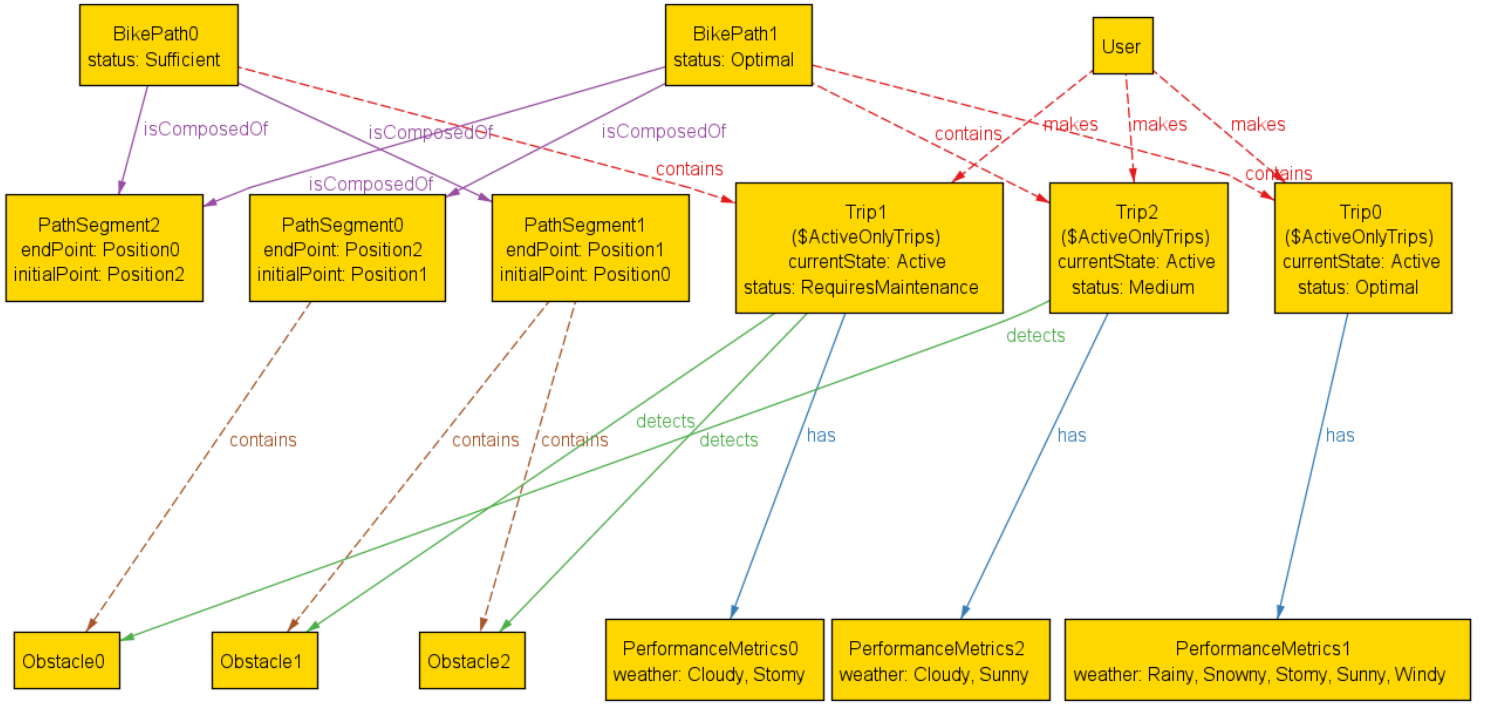


Figure 23: First step execution *ShowPublishRecord*

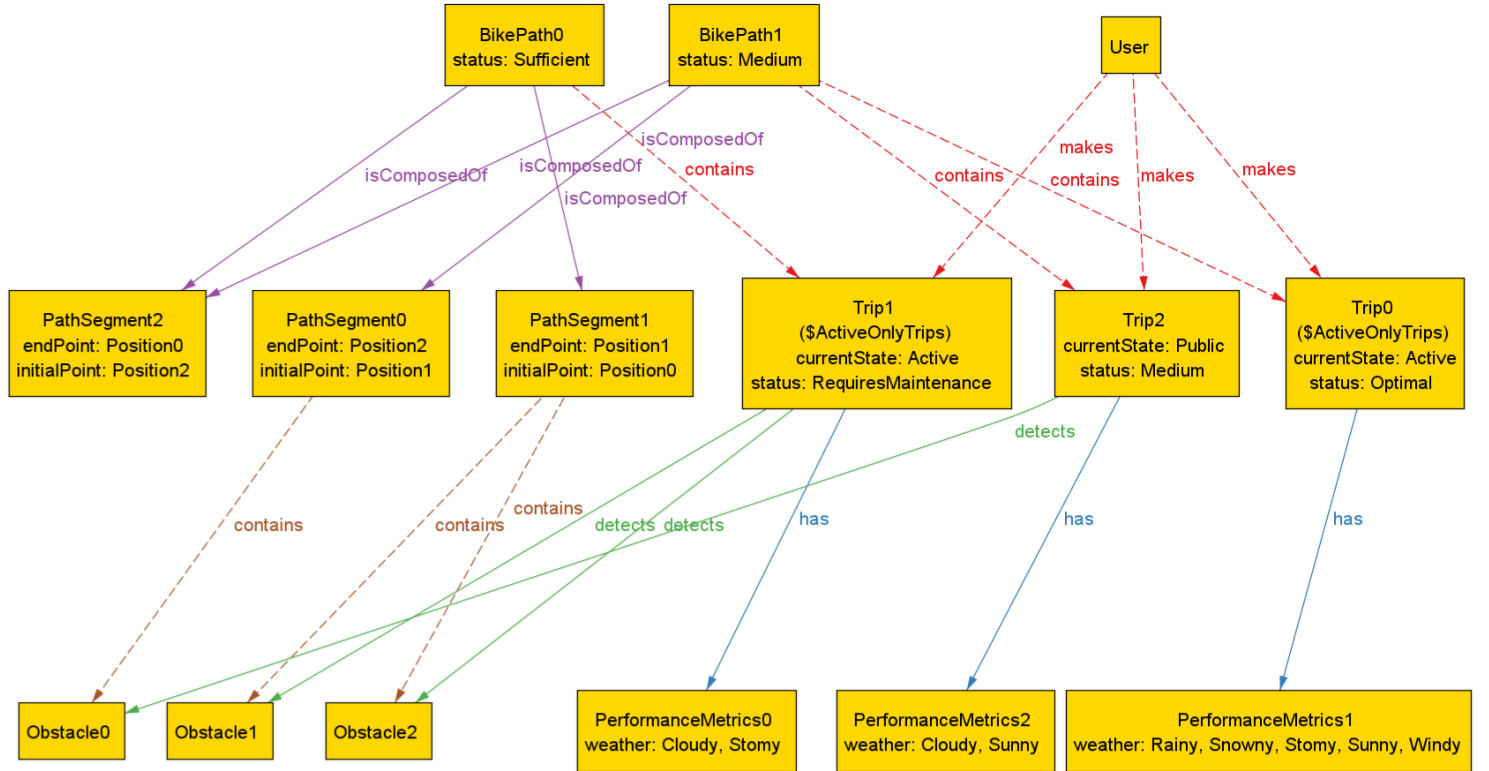


Figure 24: Second step execution *ShowPublishRecord*

- The execution of the predicate *ShowChangeStatusBikePaths* begins from a model with two users: one has a recorded trip, while the other has no recorded trips. In addition, there is a trip that has not yet been recorded.

In the first iteration (Figure 26), the recorded trip becomes public, which changes the status of the corresponding bike path. In the second iteration (Figure 27), the previously unused user completes a trip on *bikepath0* and records it (*trip0*). Finally, in the Figure 28 , the new trip is published, and the status of *bikepath0* is updated again.

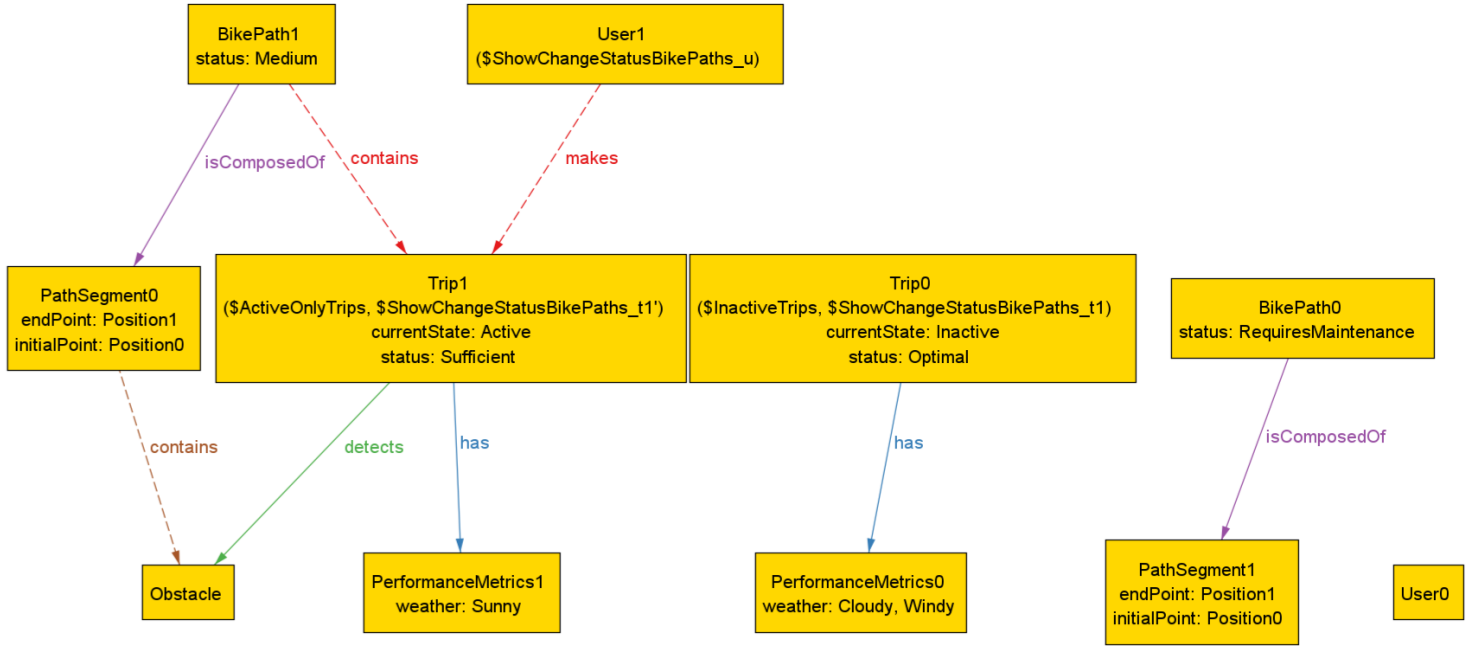


Figure 25: First step execution *ShowChangeStatusBikePaths*

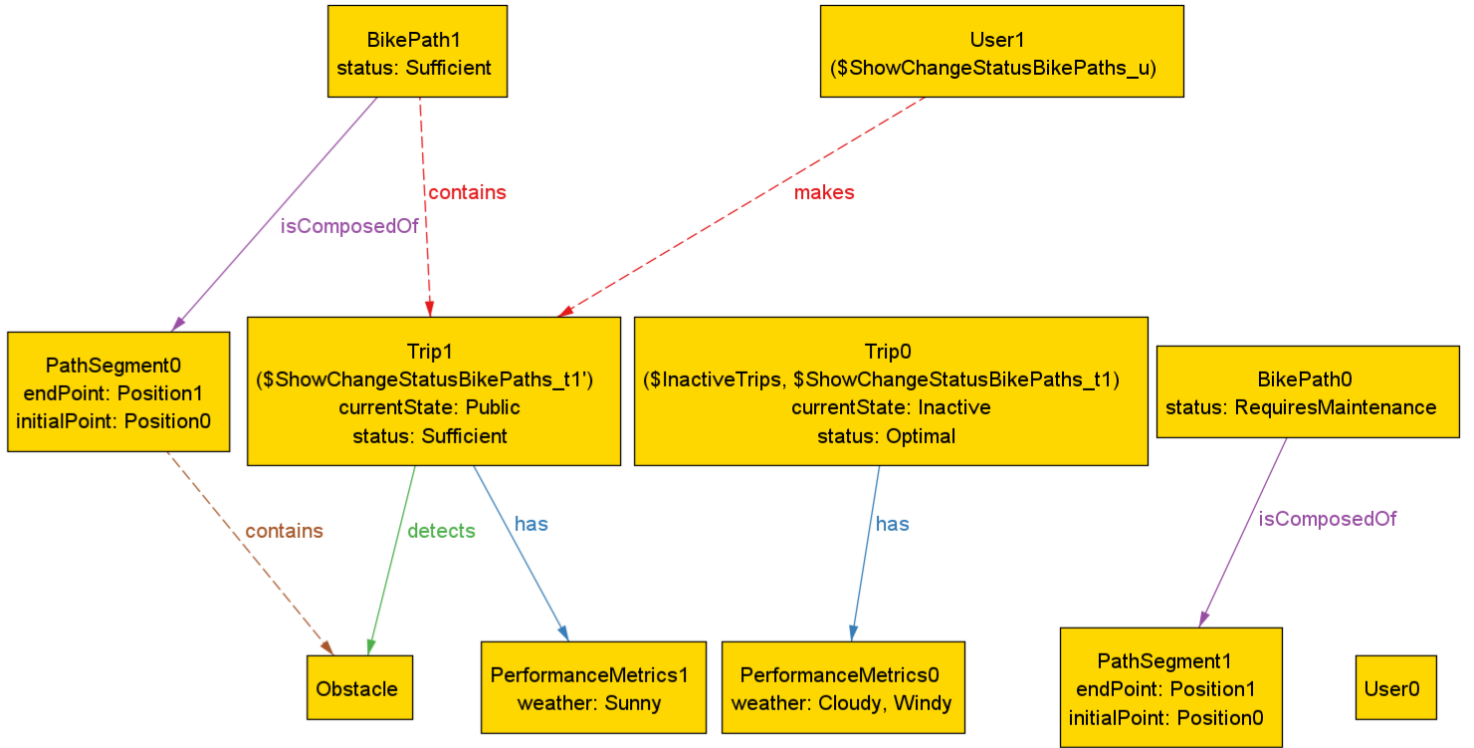


Figure 26: Second step execution *ShowChangeStatusBikePaths*

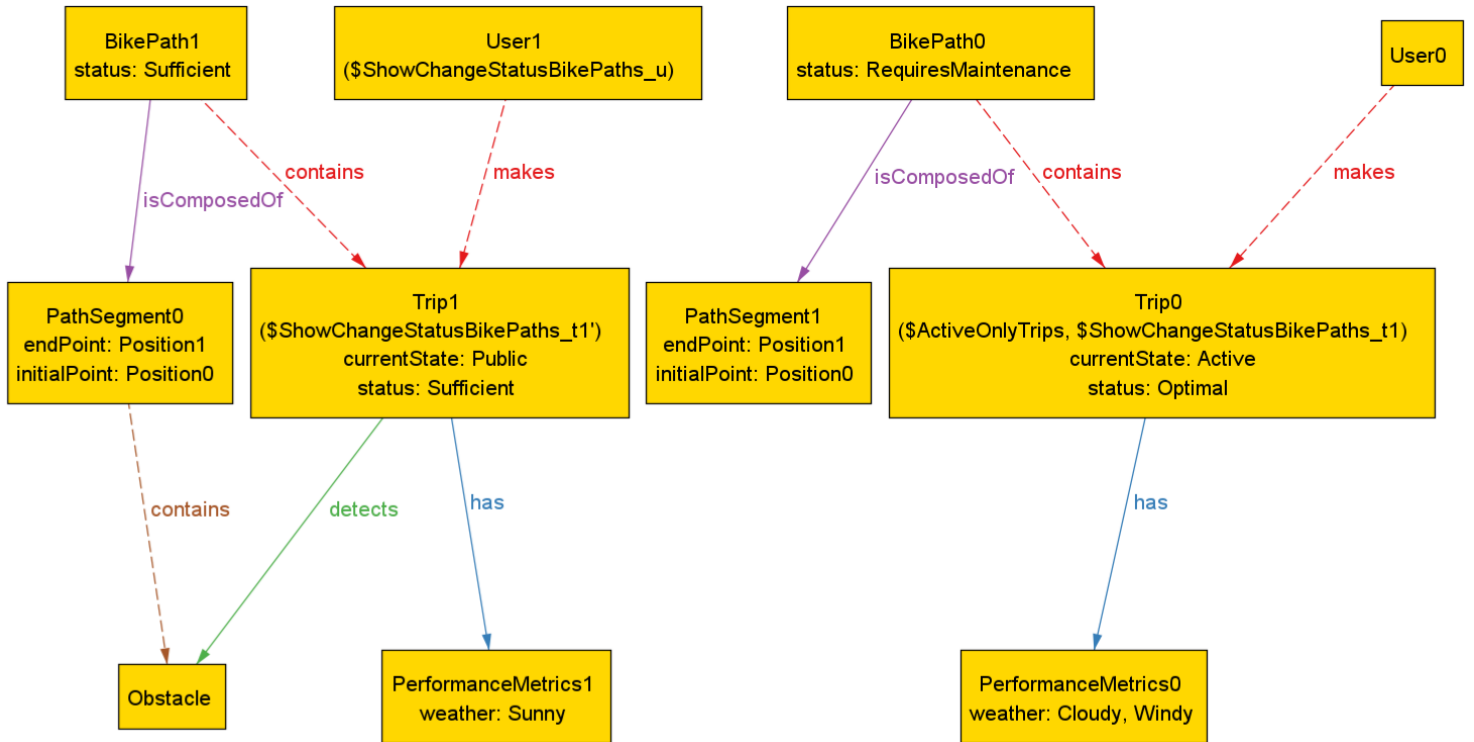


Figure 27: Third step execution *ShowChangeStatusBikePaths*

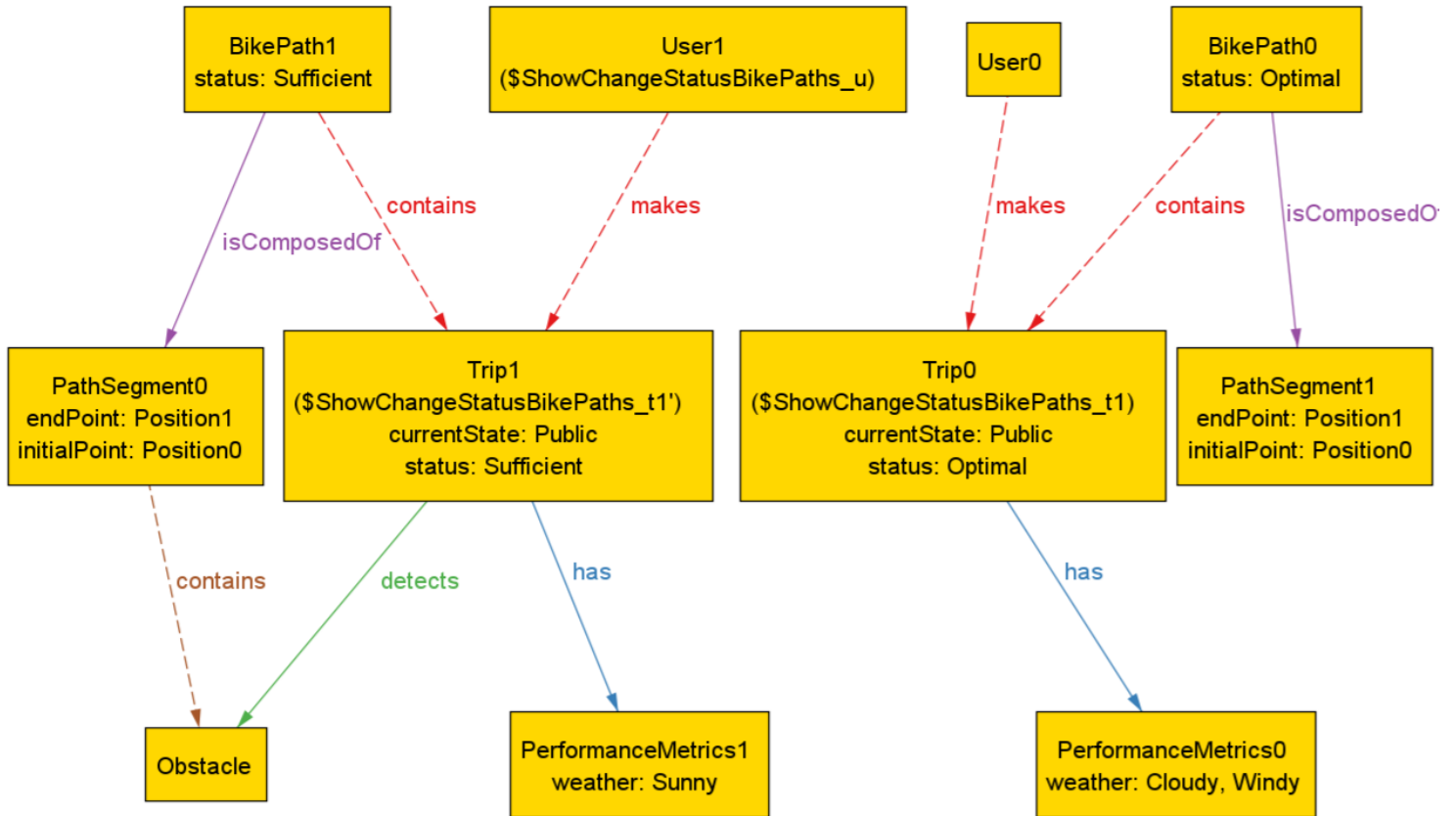


Figure 28: Fourth step execution *ShowChangeStatusBikePaths*

- By running the world simulation, we obtain the same results as in the execution of the predicate *ShowChangeStatusBikePaths*, but with additional iterations of recording and publishing actions.

5 EFFORT SPENT

Hours worked	Section 1	Section 2	Section 3	Section 4
Juan Sebastian Sarmiento Pulido	5	6	20	18
Pedro Augusto Villarinho dos Santos	4	22	4	2
Esteban Lopez Barreto	10	4	18	2

6 REFERENCES

1. <https://www.uml-diagrams.org/use-case-reference.html>
2. <https://developer.ibm.com/articles/the-sequence-diagram/>